



S81772

Don't Leave Tensors on the Table: Programming and Optimizing Tensor Cores

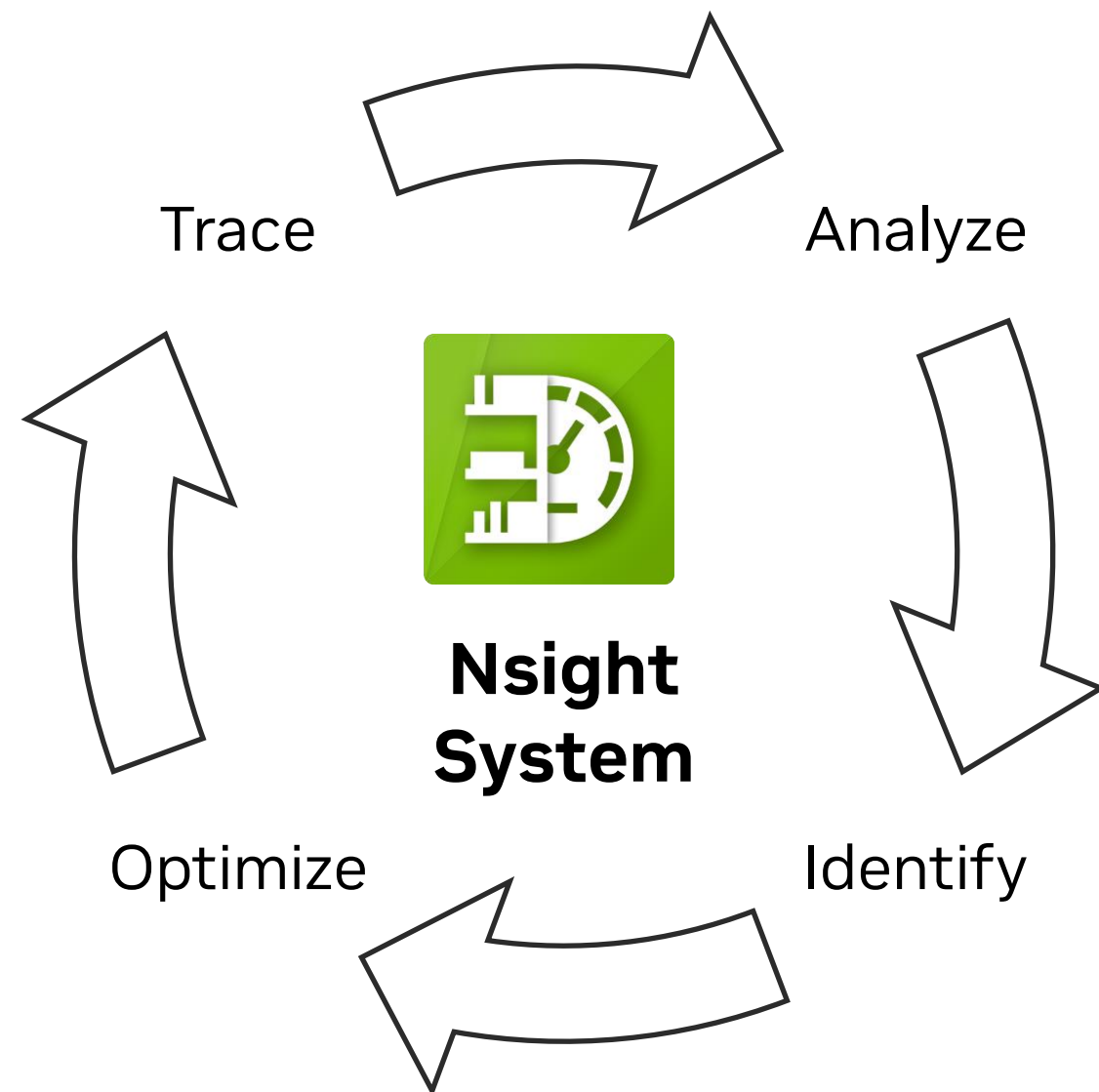
Magnus Strengert

GTC2026 | 03/19/26

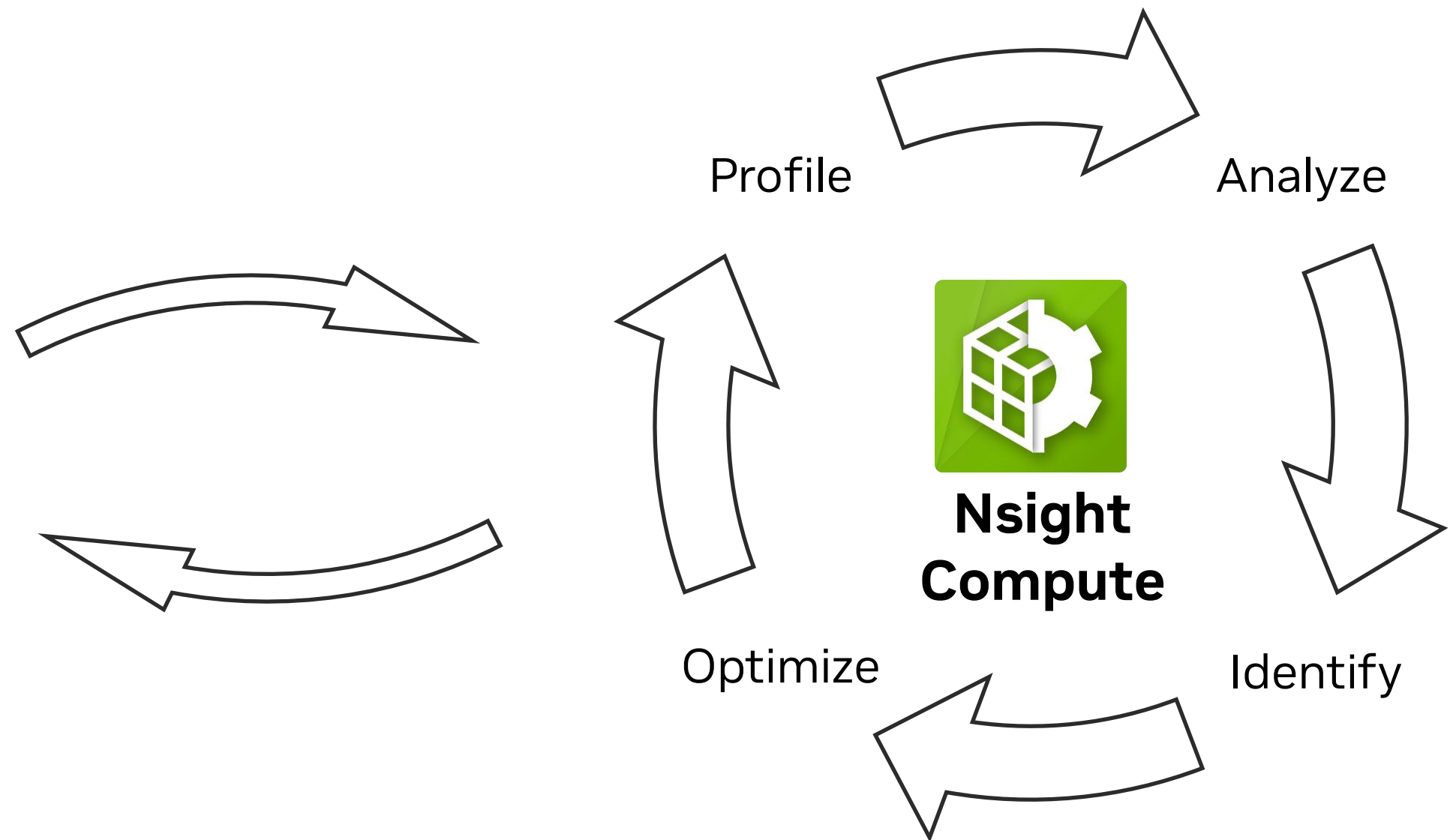


Nsight Performance Tools

Optimization Workflow



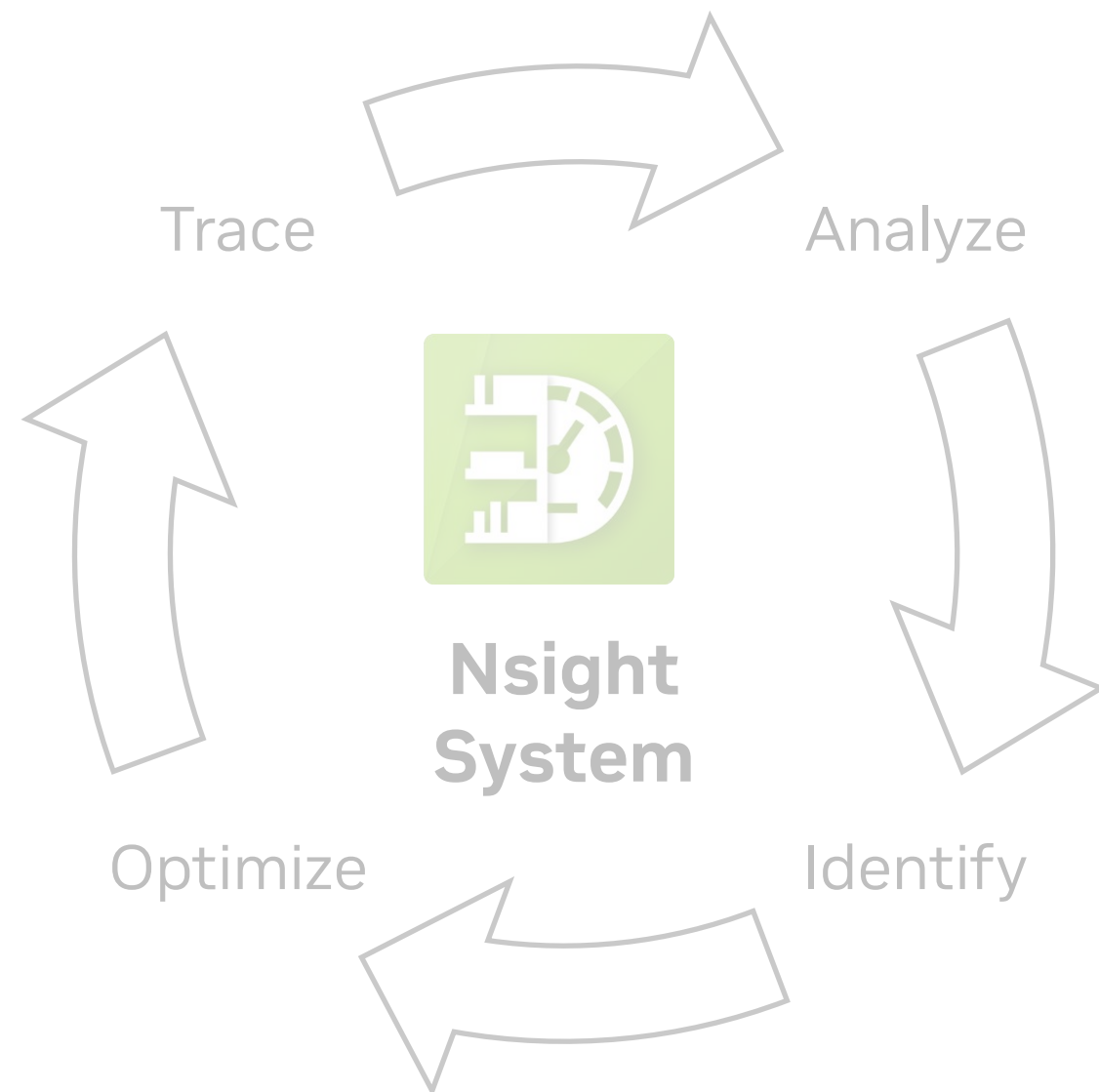
System-Level Perf Analysis



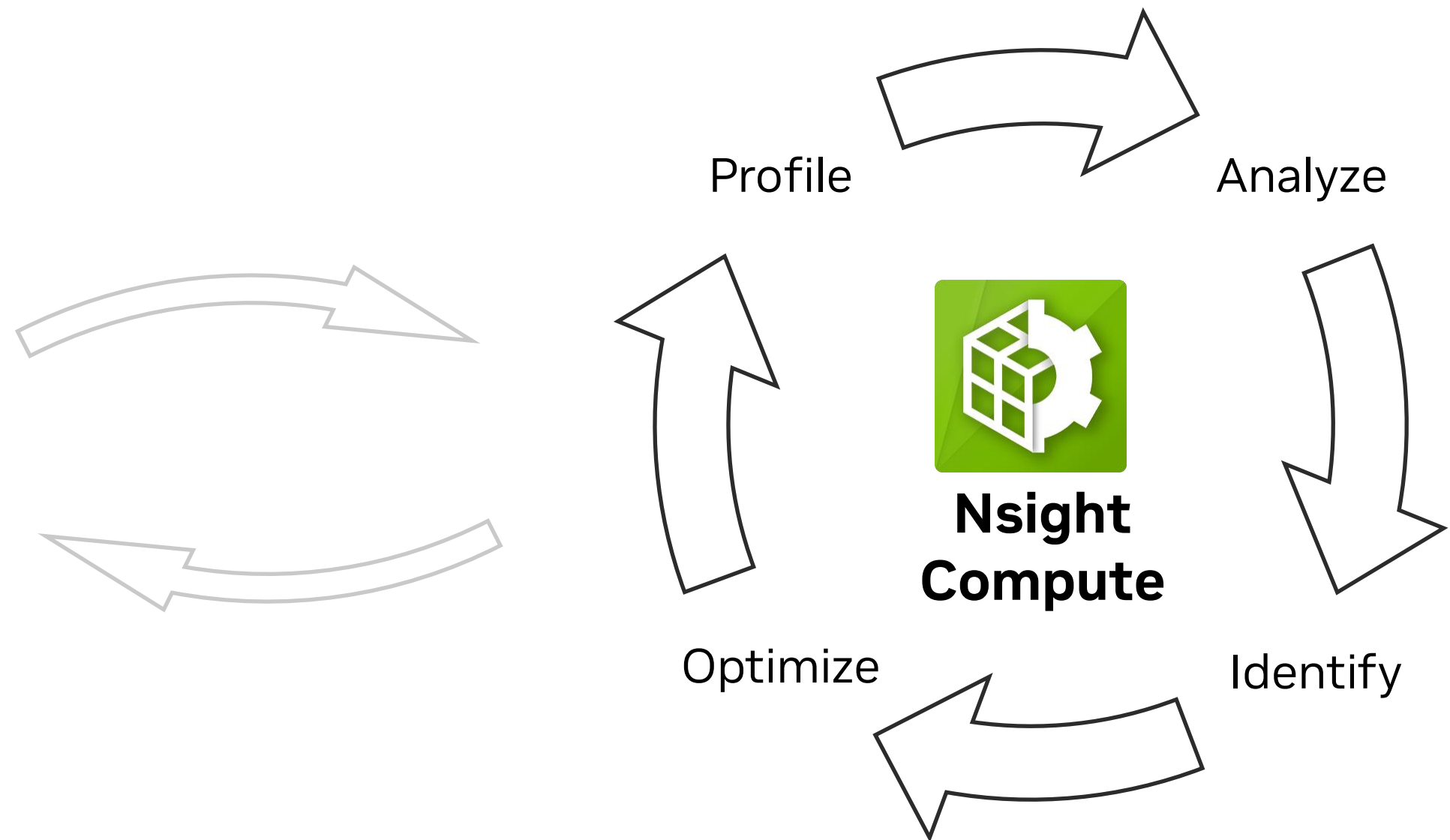
Kernel/Range-Level Perf Analysis

Nsight Performance Tools

Optimization Workflow



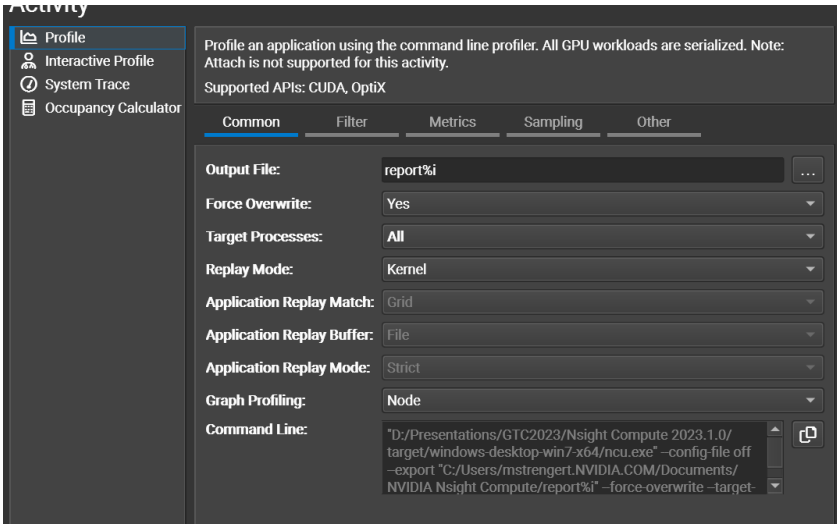
System-Level Perf Analysis



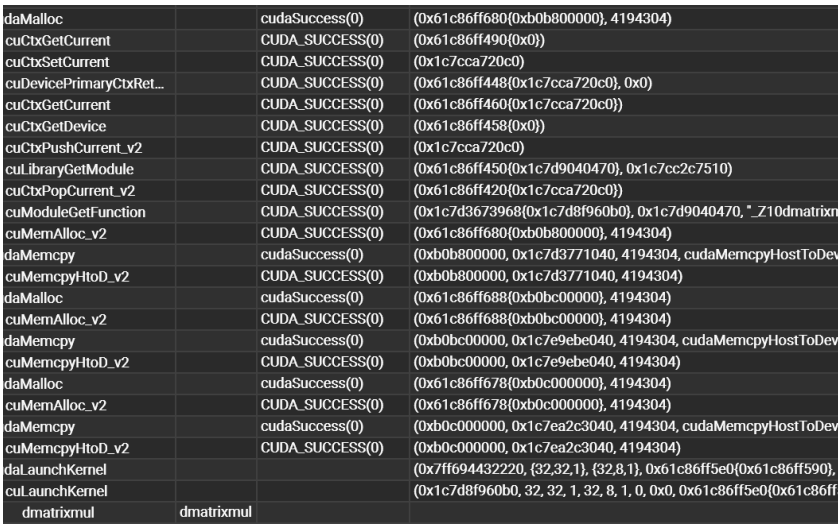
Kernel/Range-Level Perf Analysis

Nsight Compute

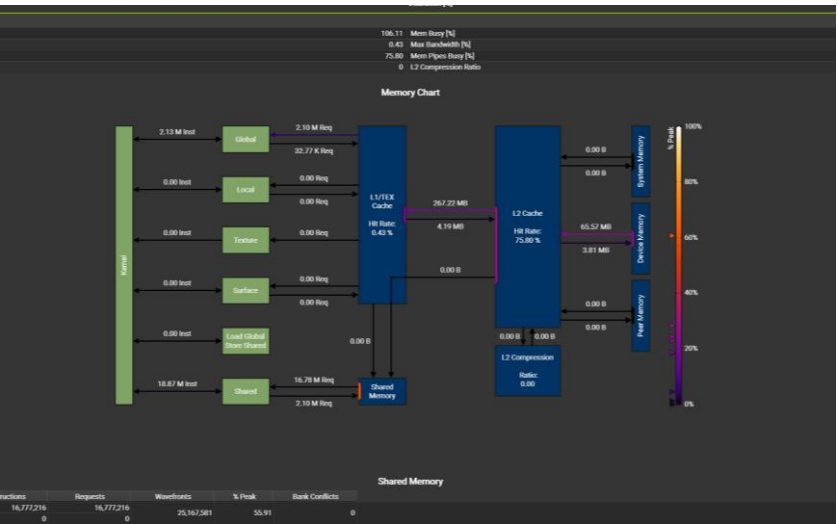
Feature Overview



GPU Kernel Profiler
Command Line and UI



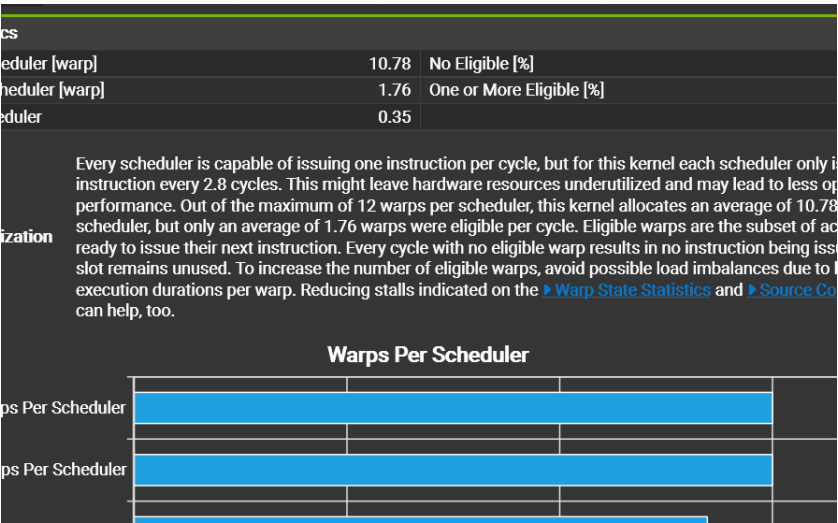
CUDA/OptiX API Stepper
With Resource Tracking / NVTX



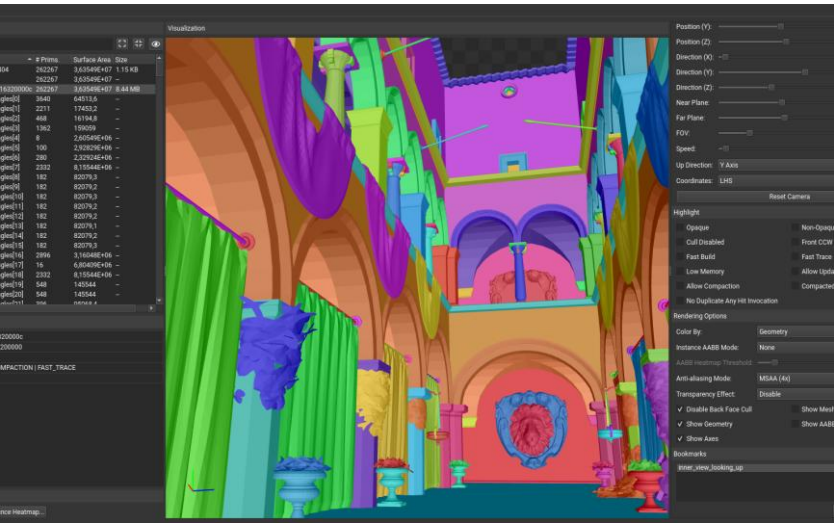
Detailed Performance Report
Metrics per Range, Graph, or Kernel



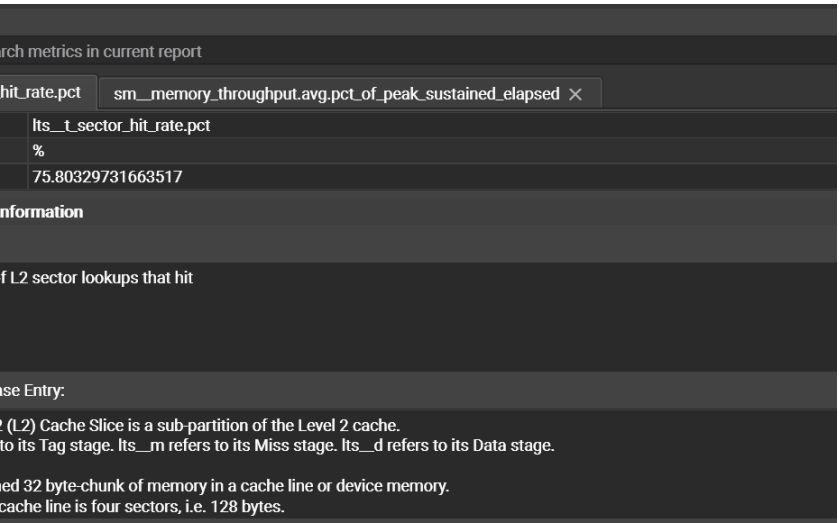
Source Correlation
Metrics and Attribution per Code Line



Guided Analysis
Issue Detection and Recommendations



OptiX Acceleration Structure
Inspection and Viewer



Documentation & Samples
Profiling Guide, Metrics Knowledge Base

NVIDIA Nsight Compute

New in CUDA 2025.4(13.1)/2026.1(13.2)

Multi-Process/Multi-Report Support

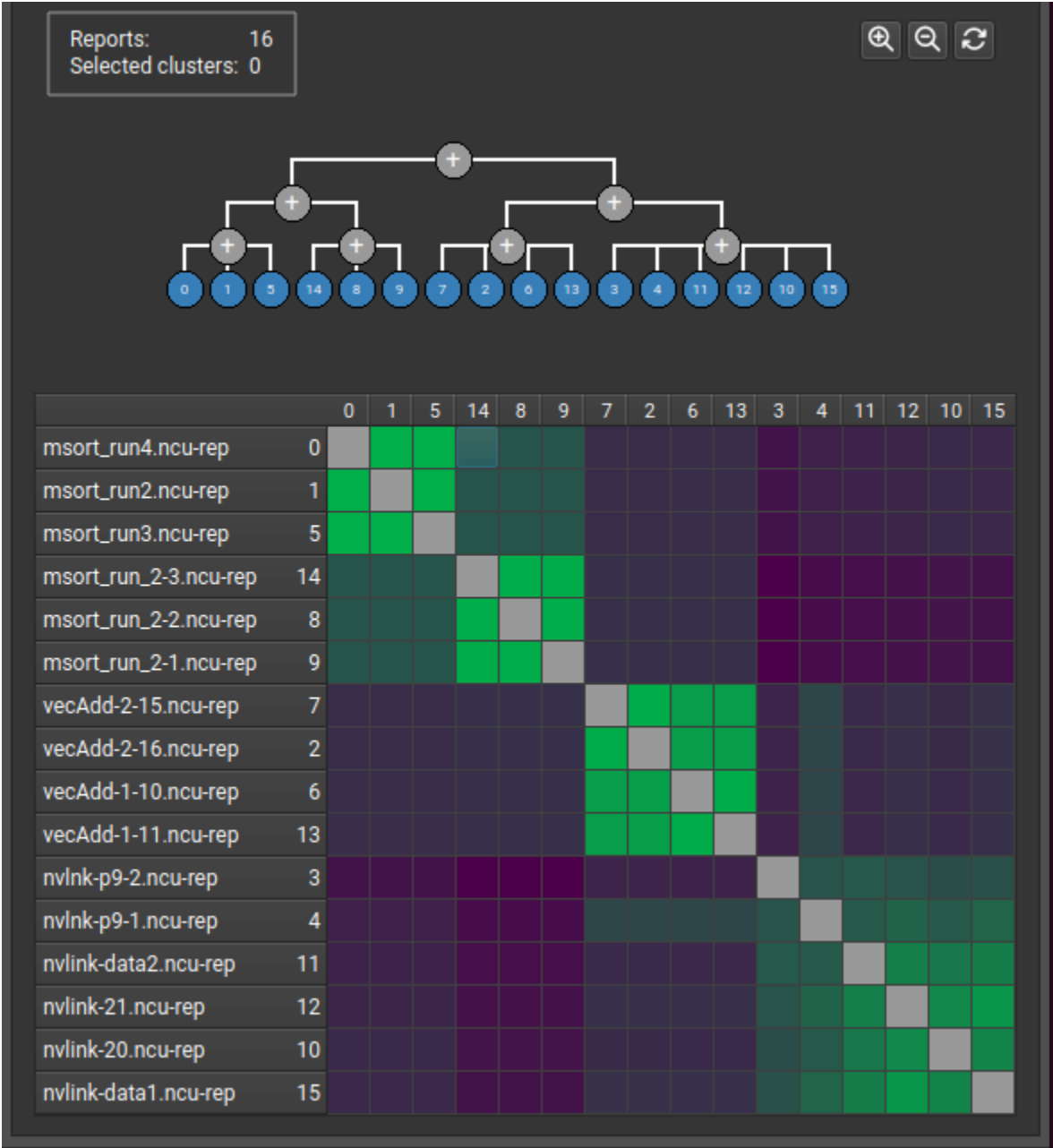
- Communicator for multi-process synchronization
 - Enables profiling of NCCL, mandatory-concurrent kernels
- Report Merge tool
- Multi-Report Analysis/Cluster

PM Sampling

- Improved Timeline Row Hierarchy (overlay rows)
- Combined PM+PC Sampling
 - Answers what code was executed at a given time on the timeline

Improvements for CUDA Graph Profiling

- Support for per-node profiling of graphs with conditional graphs and device-side graph launches
- New interactive CUDA Graph Viewer tool window



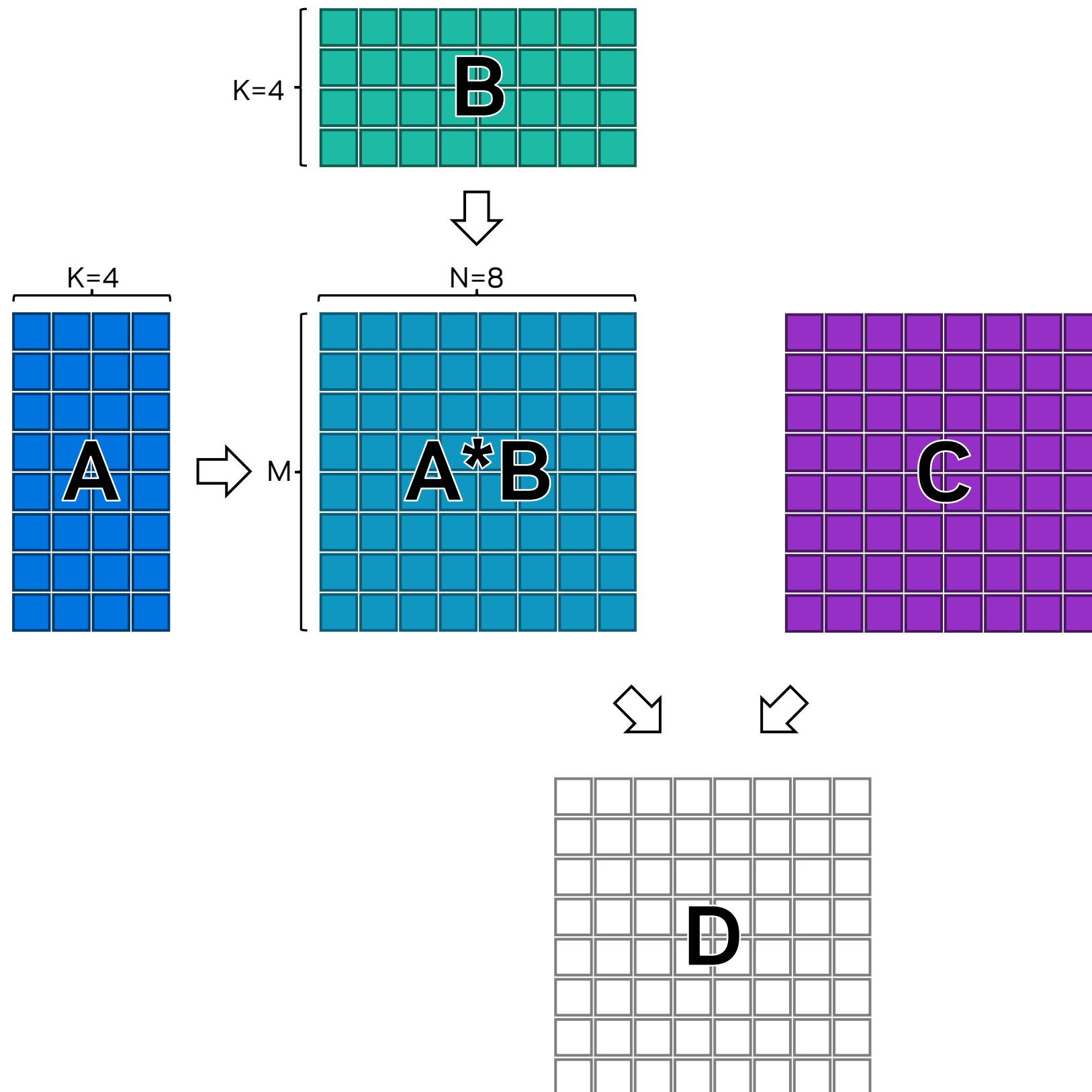
Function Stats									
Function Name	^	Samples (All)	Samples (Not Issued)	Top Stall #1	Top Stall #2	Top Stall #3			
tex2D		30%	31%	LGSB	45%	WAIT	38%	LGTHR	8%
type tex2D		28%	28%	WAIT	48%	LGSB	33%	SHRTSB	12%
SobelCopyImage		23%	22%	WAIT	63%	MEMBAR	21%	SHRTSB	7%
min		8%	8%	WAIT	55%	LGSB	25%	BRANCH	6%
max		7%	7%	WAIT	64%	LGSB	19%	BRANCH	7%
fminf		2%	2%	WAIT	54%	NOINST	23%	BRANCH	19%
fmaxf		2%	2%	WAIT	46%	BRANCH	26%	NOINST	24%

Tensor Cores: Fundamentals



Matrix Multiply-and-Accumulate (MMA)

Basics



Calculates: $D = A * B + C$

- Matrices D and C might refer to the same matrix (accumulation)
- Inputs matrices A,B may have different data types as matrices C,D

Sizes: $M \times N \times K$ (8x8x4)

- Matrix A: $M \times K$ (8x4), Matrix B: $K \times N$ (4x8), Matrix C/D: $M \times N$ (8x8)

Basic Algorithm:

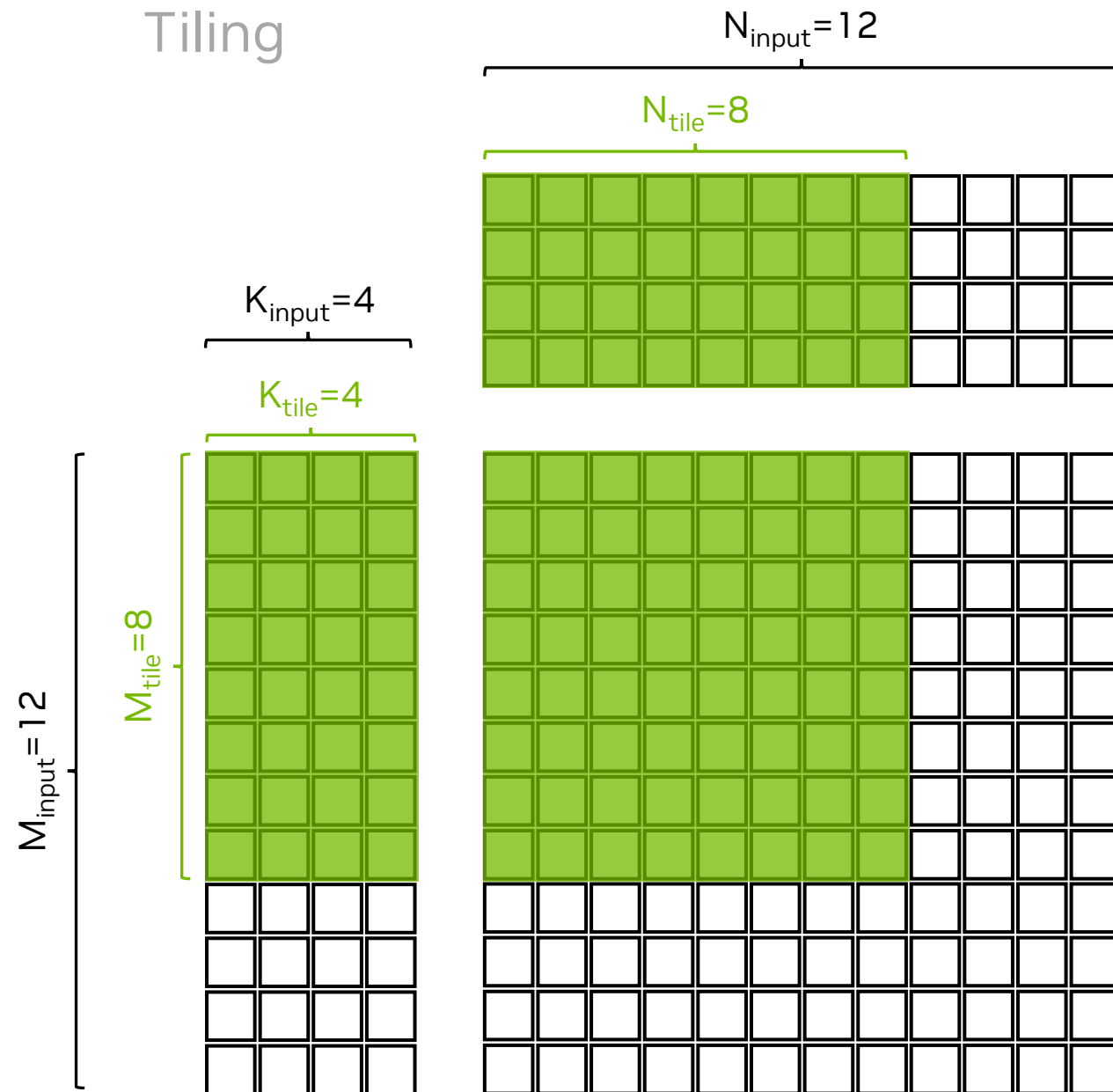
```
for m in [0, M):  
    for n in [0, N):  
        D[n,m] = C[n,m]  
        for k in [0, K):  
            D[n,m] += A[n,k] * B[m, j]
```

FLOPS:

- Total: $2 \times M \times N \times K$ ($2 \times 8 \times 8 \times 4 = 512$) operations
 - $M \times N \times K$ (256) multiplications and $M \times N \times K$ (256) additions
 - $M \times N \times K$ (256) fused-multiply-add operations

Matrix Multiply-and-Accumulate (MMA)

Tiling



GPU hardware support specific MxNxK sizes

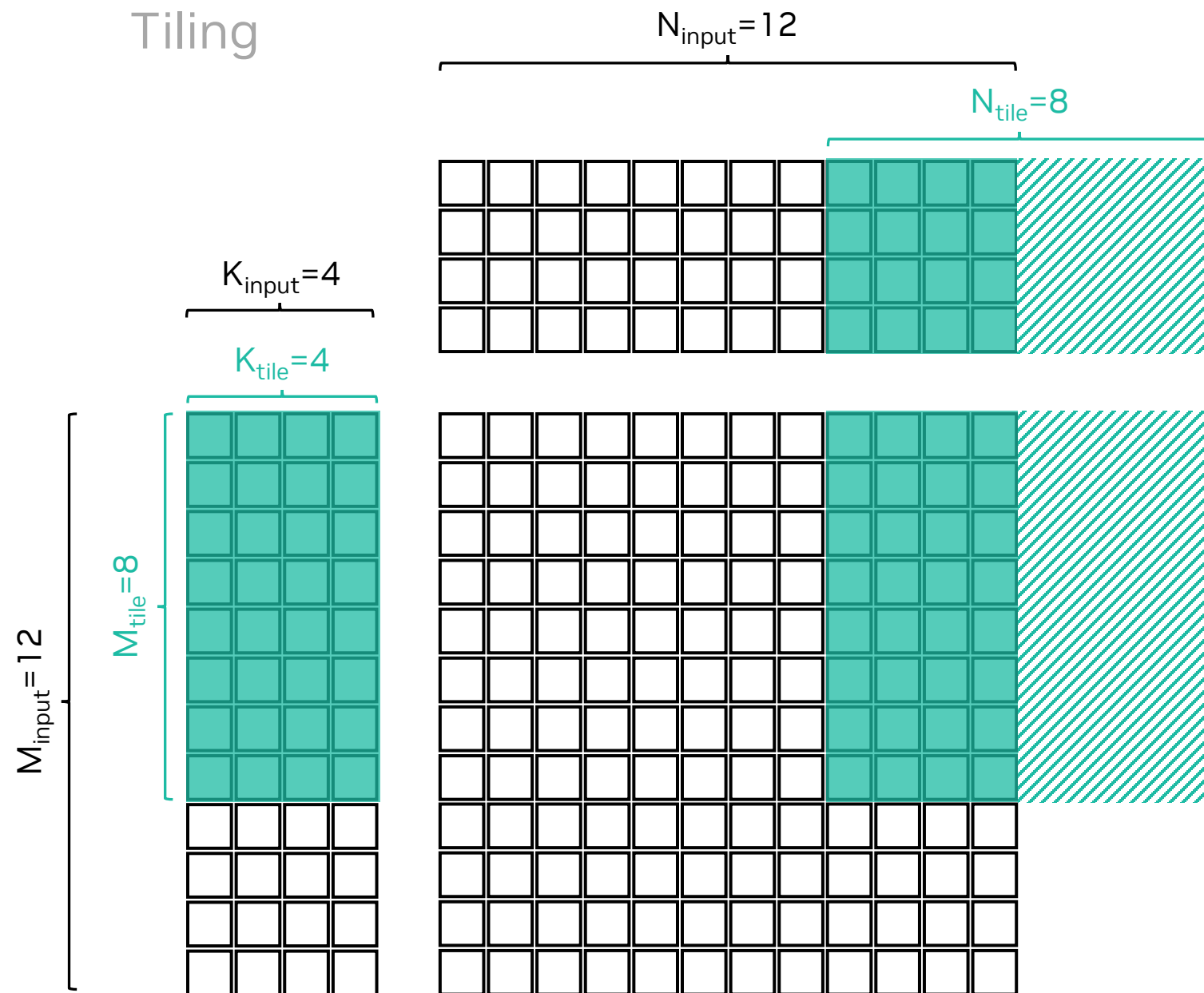
- Exact sizes differ by: architecture, format, mma-type
- Full details at:
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>

Tiling used for all other sizes

- Attempts to select optimal balance between
 - High efficiency (larger MMA sizes)
 - Low waste (smaller MMA sizes)

Matrix Multiply-and-Accumulate (MMA)

Tiling



GPU hardware support specific MxNxK sizes

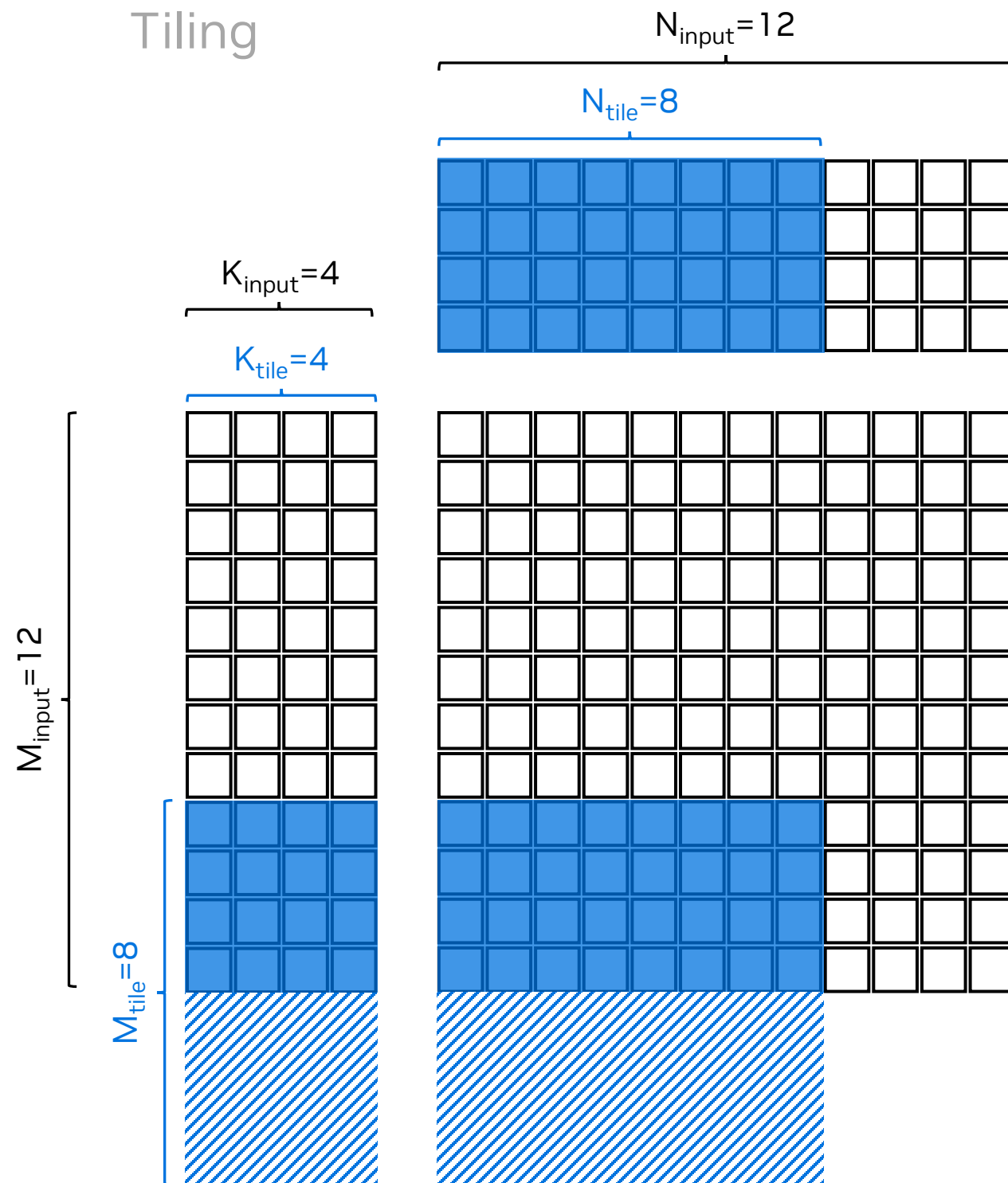
- Exact sizes differ by: architecture, format, mma-type
- Full details at:
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>

Tiling used for all other sizes

- Attempts to select optimal balance between
 - High efficiency (larger MMA sizes)
 - Low waste (smaller MMA sizes)

Matrix Multiply-and-Accumulate (MMA)

Tiling



GPU hardware support specific MxNxK sizes

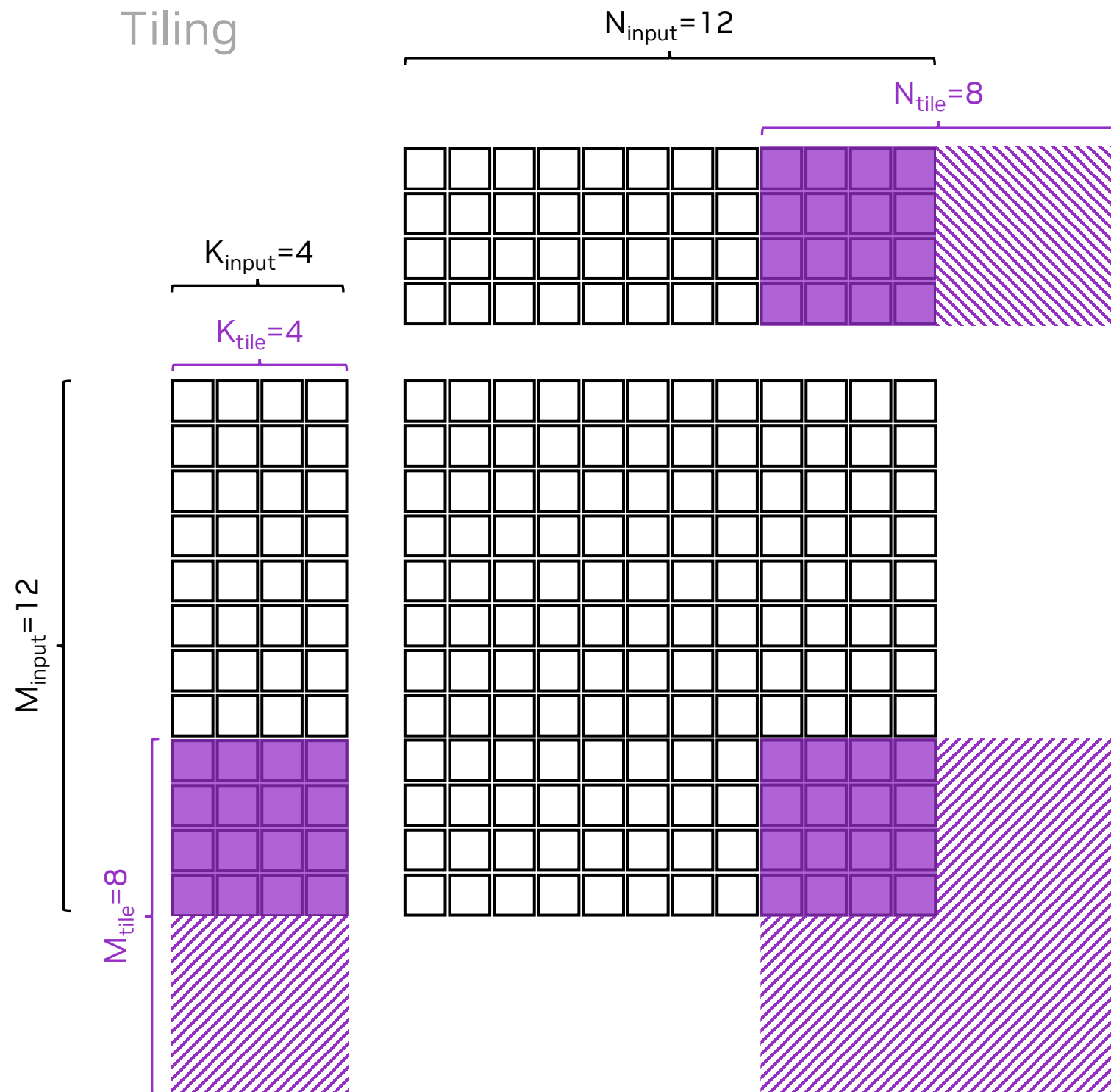
- Exact sizes differ by: architecture, format, mma-type
- Full details at:
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>

Tiling used for all other sizes

- Attempts to select optimal balance between
 - High efficiency (larger MMA sizes)
 - Low waste (smaller MMA sizes)

Matrix Multiply-and-Accumulate (MMA)

Tiling



GPU hardware support specific MxNxK sizes

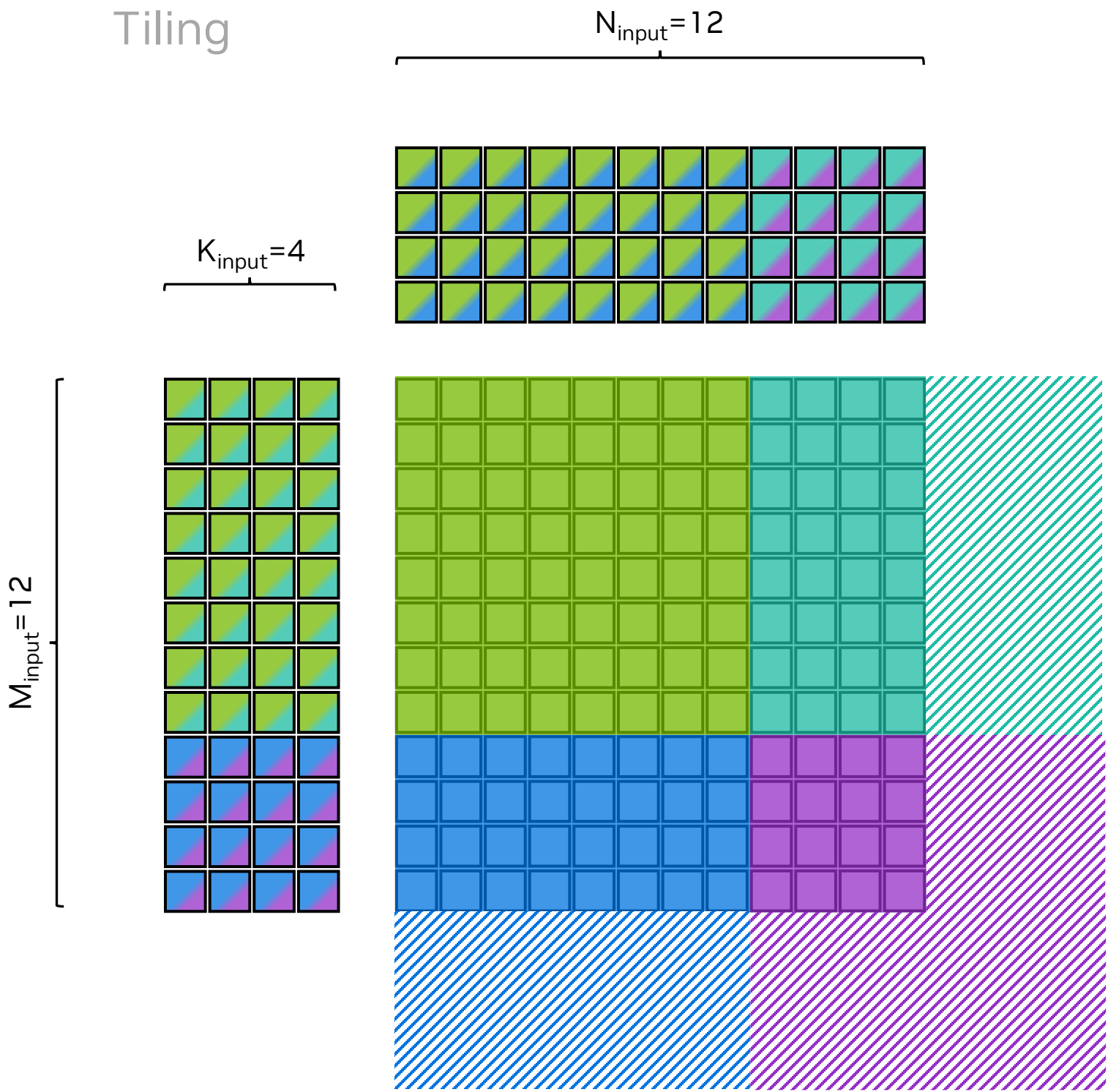
- Exact sizes differ by: architecture, format, mma-type
- Full details at:
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>
 - <https://docs.nvidia.com/cuda/parallel-thread-execution/#asynchronous-warpgroup-level-matrix-shape>

Tiling used for all other sizes

- Attempts to select optimal balance between
 - High efficiency (larger MMA sizes)
 - Low waste (smaller MMA sizes)

Matrix Multiply-and-Accumulate (MMA)

Tiling

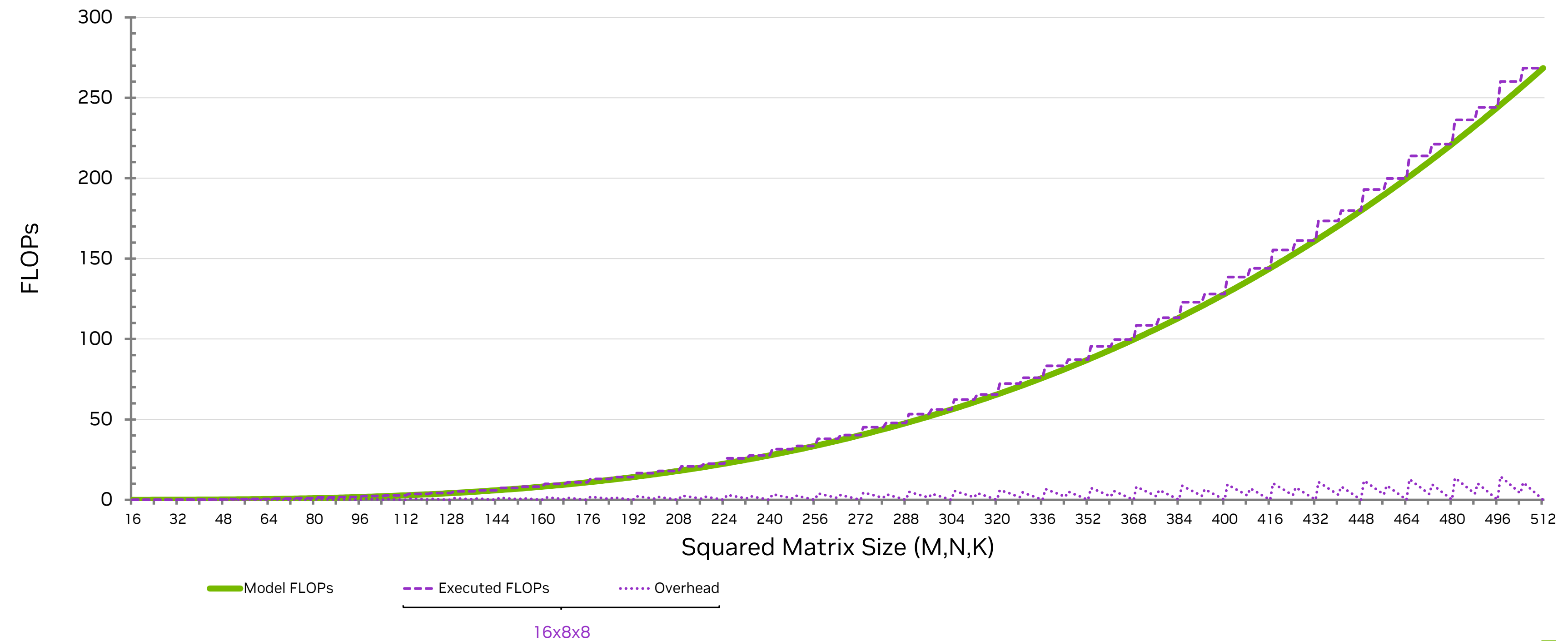


Overhead Estimation (Matrix Multiplication only):

• Input:	$M_{input} \times N_{input} \times K_{input}$	12x12x4
• Tile:	$M_{tile} \times N_{tile} \times K_{tile}$	8x8x4
• Tile Count:	$\text{ceil}(M_{input}/M_{tile}) \times \text{ceil}(N_{input}/N_{tile}) \times \text{ceil}(K_{input}/K_{tile})$	2x2x1
• Model FLOPs:	$M_{input} \times N_{input} \times (2 * K_{input} - 1)$	1008 ops
• Executed FLOPs:	$M_{tile} \times N_{tile} \times (2 * K_{tile} - 1) \times \text{TileCount}$	1792 ops
• Overhead:	Executed FLOPs - Model FLOPs	784 ops
• Memory Accesses:		
• Matrix A:	$\text{ceil}(M_{input}/M_{tile})$	2 times
• Matrix B:	$\text{ceil}(N_{input}/N_{tile})$	2 times
• Output:	$\text{ceil}(K_{input}/K_{tile})$	1 time

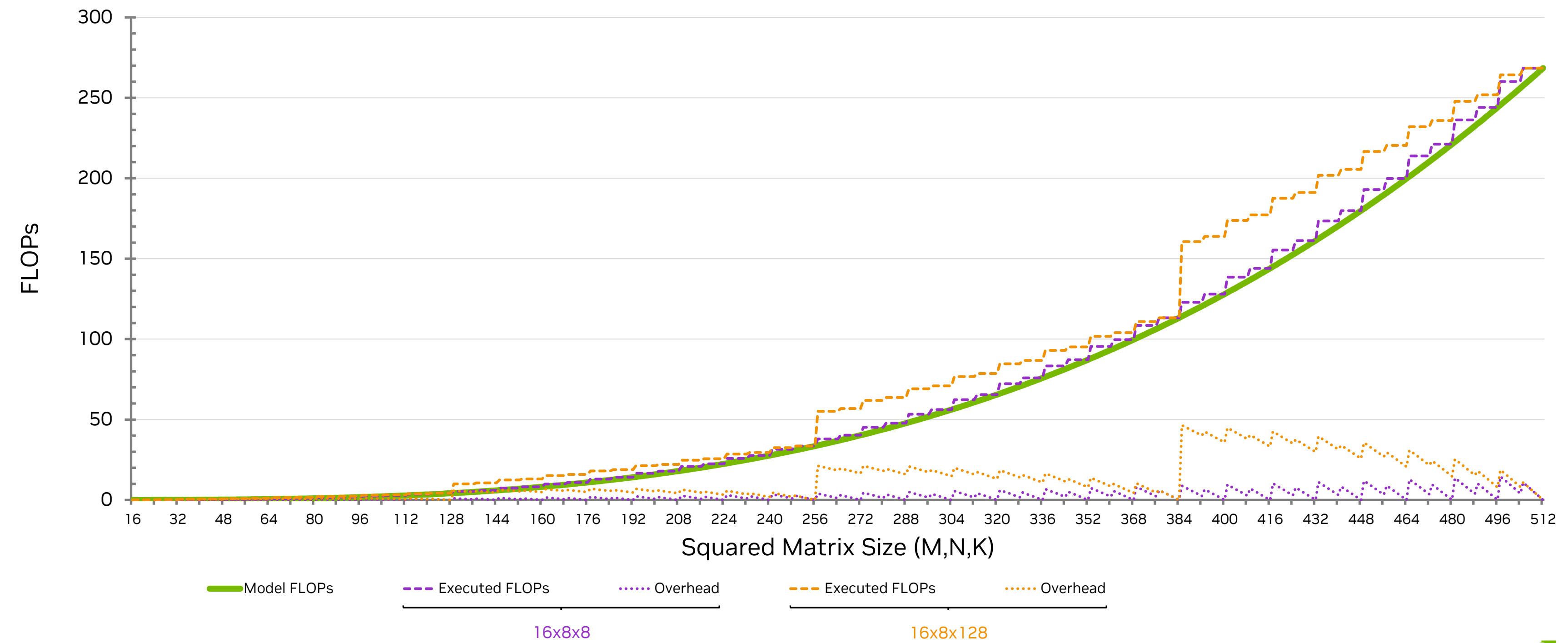
Matrix Multiply-and-Accumulate (MMA)

Tiling Overhead



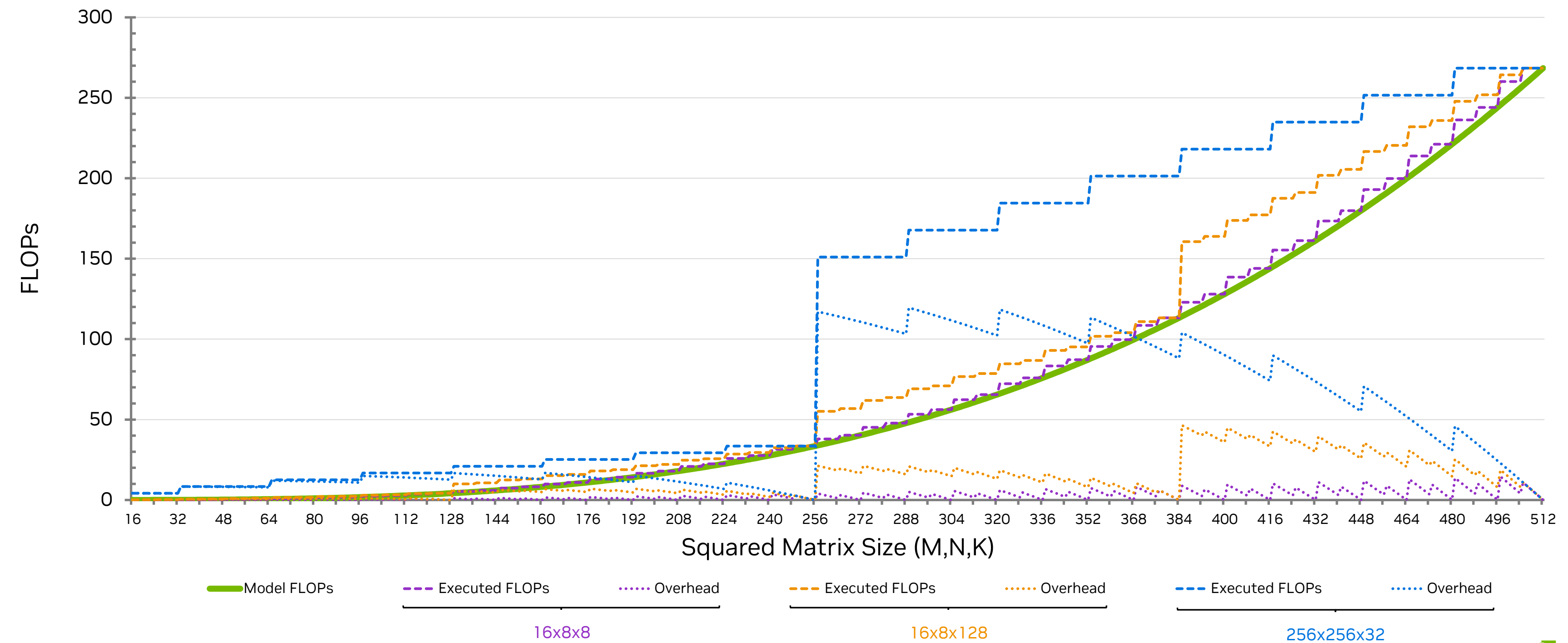
Matrix Multiply-and-Accumulate (MMA)

Tiling Overhead



Matrix Multiply-and-Accumulate (MMA)

Tiling Overhead



Tensor Cores: SM Mental Model & Tensor Op Types



Streaming Multiprocessor (SM)

Mental Model



- Warp Scheduler , each with:
- Issue Stage (1 instruction/cycle)

Streaming Multiprocessor (SM)

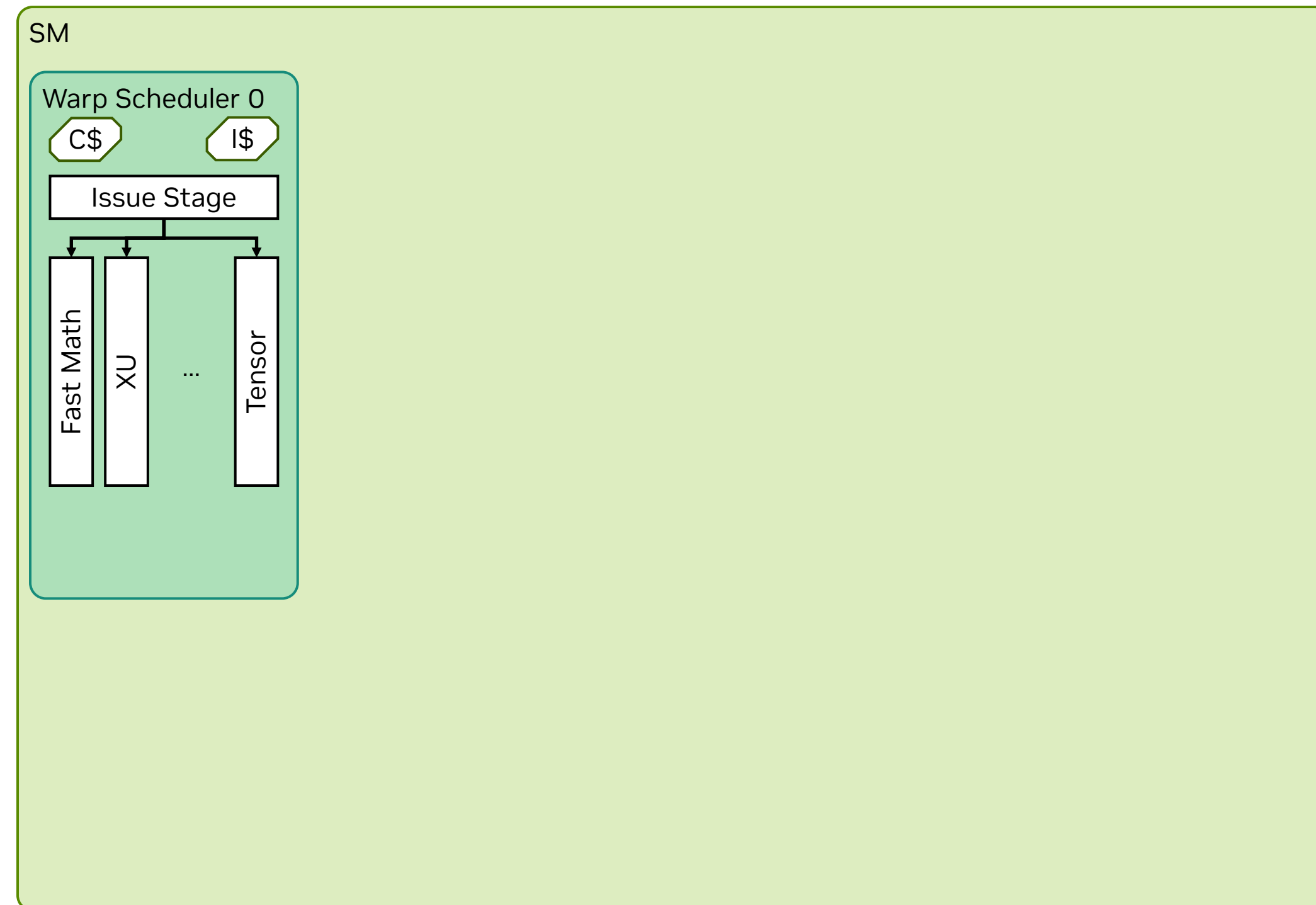
Mental Model



- Warp Scheduler , each with:
- Issue Stage (1 instruction/cycle)
- Instruction Cache (I\$), Constant Cache (C\$)

Streaming Multiprocessor (SM)

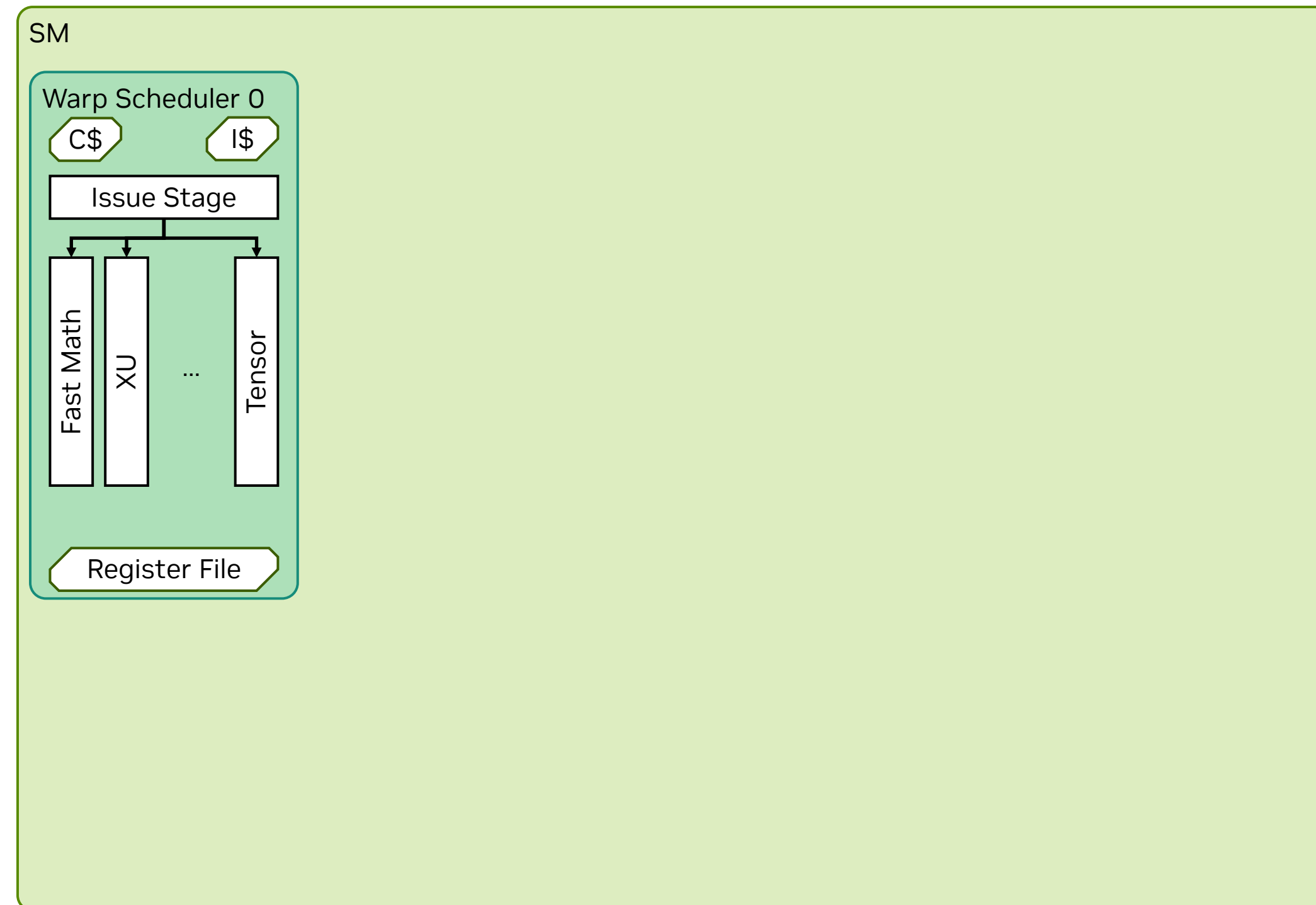
Mental Model



- Warp Scheduler , each with:
 - Issue Stage (1 instruction/cycle)
 - Instruction Cache (I\$), Constant Cache (C\$)
 - Individual Execution Pipelines
 - Fast Match (1 FMA/cycle)
 - Tensor

Streaming Multiprocessor (SM)

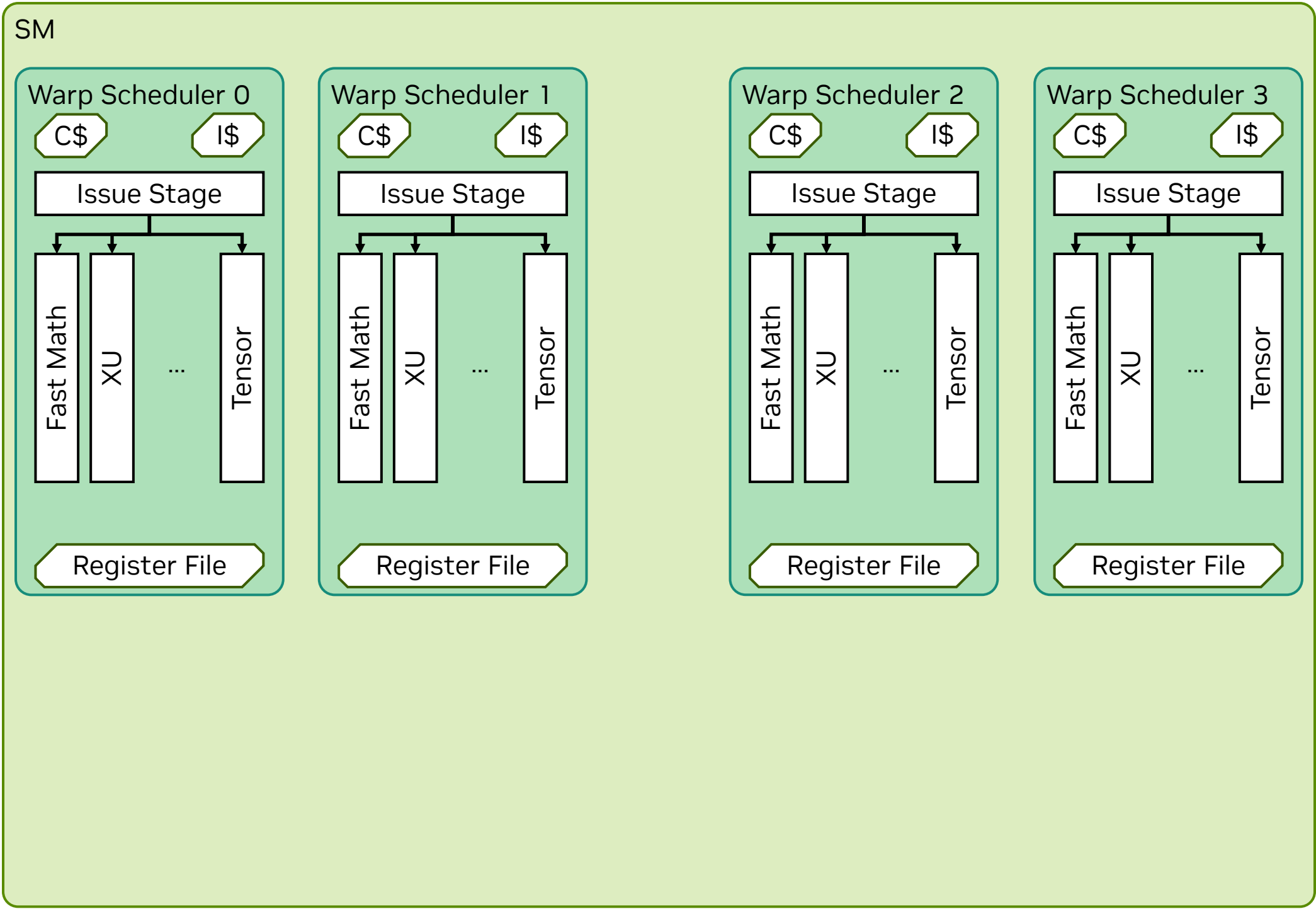
Mental Model



- Warp Scheduler , each with:
 - Issue Stage (1 instruction/cycle)
 - Instruction Cache (I\$), Constant Cache (C\$)
 - Individual Execution Pipelines
 - Fast Match (1 FMA/cycle)
 - Tensor
 - Register File (64k bytes)

Streaming Multiprocessor (SM)

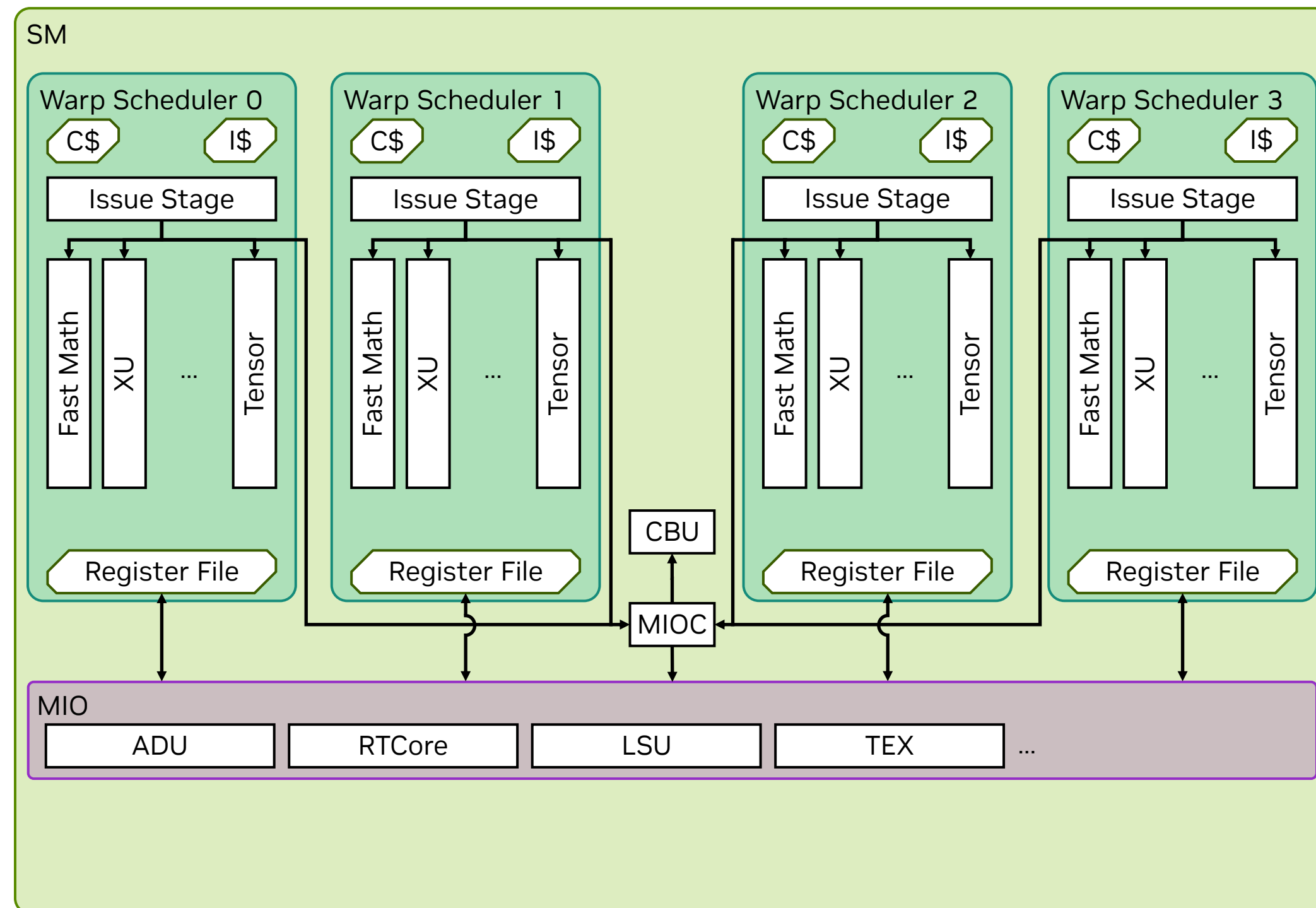
Mental Model



- 4 Warp Schedulers, each with:
 - Issue Stage (1 instruction/cycle)
 - L0 Instruction Cache (I\$), Constant Cache (C\$)
 - Individual Execution Pipelines
 - Fast Math (1 FMA/cycle)
 - Tensor
 - Register File (64k bytes)

Streaming Multiprocessor (SM)

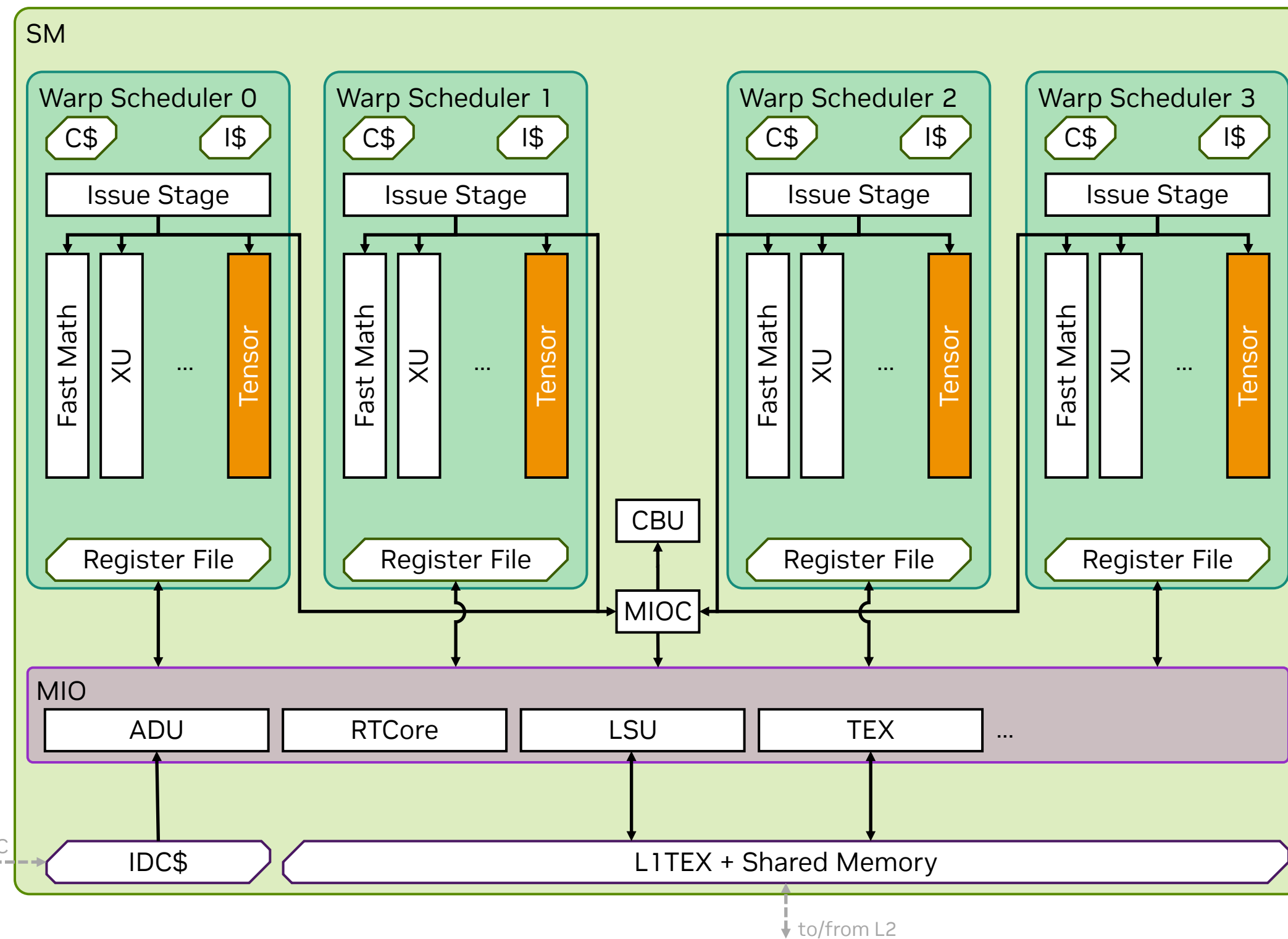
Mental Model



- 4 Warp Schedulers, each with:
 - Issue Stage (1 instruction/cycle)
 - L0 Instruction Cache (I\$), Constant Cache (C\$)
 - Individual Execution Pipelines
 - Fast Match (1 FMA/cycle)
 - Tensor
 - Register File (64k bytes)
- Memory Input/Output (MIO)
 - Shared Memory IO Controller (MIOC)
 - Shared Execution Pipelines

Streaming Multiprocessor (SM)

Mental Model



- 4 Warp Schedulers, each with:
 - Issue Stage (1 instruction/cycle)
 - L0 Instruction Cache (I\$), Constant Cache (C\$)
 - Individual Execution Pipelines
 - Fast Match (1 FMA/cycle)
 - **Tensor**
 - Register File (64k bytes)
- Memory Input/Output (MIO)
 - Shared Memory IO Controller (MIOC)
 - Shared Execution Pipelines
- Local Caches:
 - L1TEX / Shared Memory
 - Indexed Constant Cache (IDC\$)

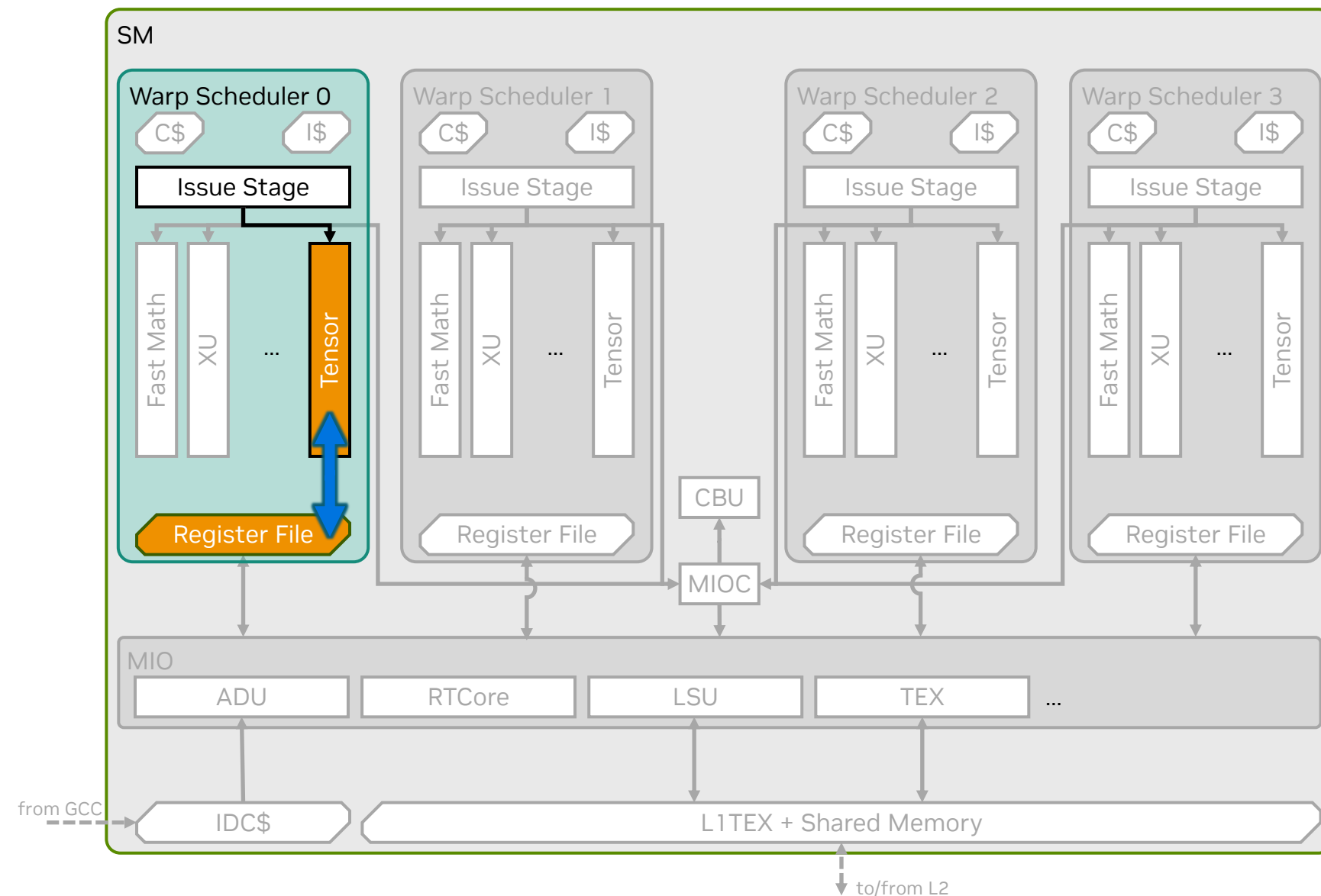
Tensor Cores

Operation Types

	Warp	Warpgroup	TensorCore 5 th Gen
Supported By	Volta and all newer	Hopper GH100/SM_90a	Blackwell GB100/SM_100a/SM100f, GB10b/SM110a/SM_110f
Execution Granularity	Warp All threads in the warp issue	Warpgroup (4 warps working together) All threads in the warpgroup issue	1 Warpgroup or 2 Warpgroups, Only 1 thread issues across the warpgroup(s)
PTX Exposure	wmma, mma	wgmma	tcgen05.mma
Format	$D = A * B + C$ $D = A * B$	$D = A * B + C$ $D = A * B$	$D = A * B + D$ $D = A * B$
Sources	A, B, C, D in register file (of 1 warp scheduler)	A, B from shared memory C, D in register file (of 4 warp schedulers)	A from TMEM or shared memory B from shared memory D in Tensor Core Memory (TMEM)
Size Range (fp16)	8x8x4 .. 16x8x16	64x8x16 .. 64x256x16	1CTA: 64x8x16 .. 128x256x16 2CTA: 128x16x16 .. 256x256x16

Warp Level MMA Instructions

Overview



Overall Idea:

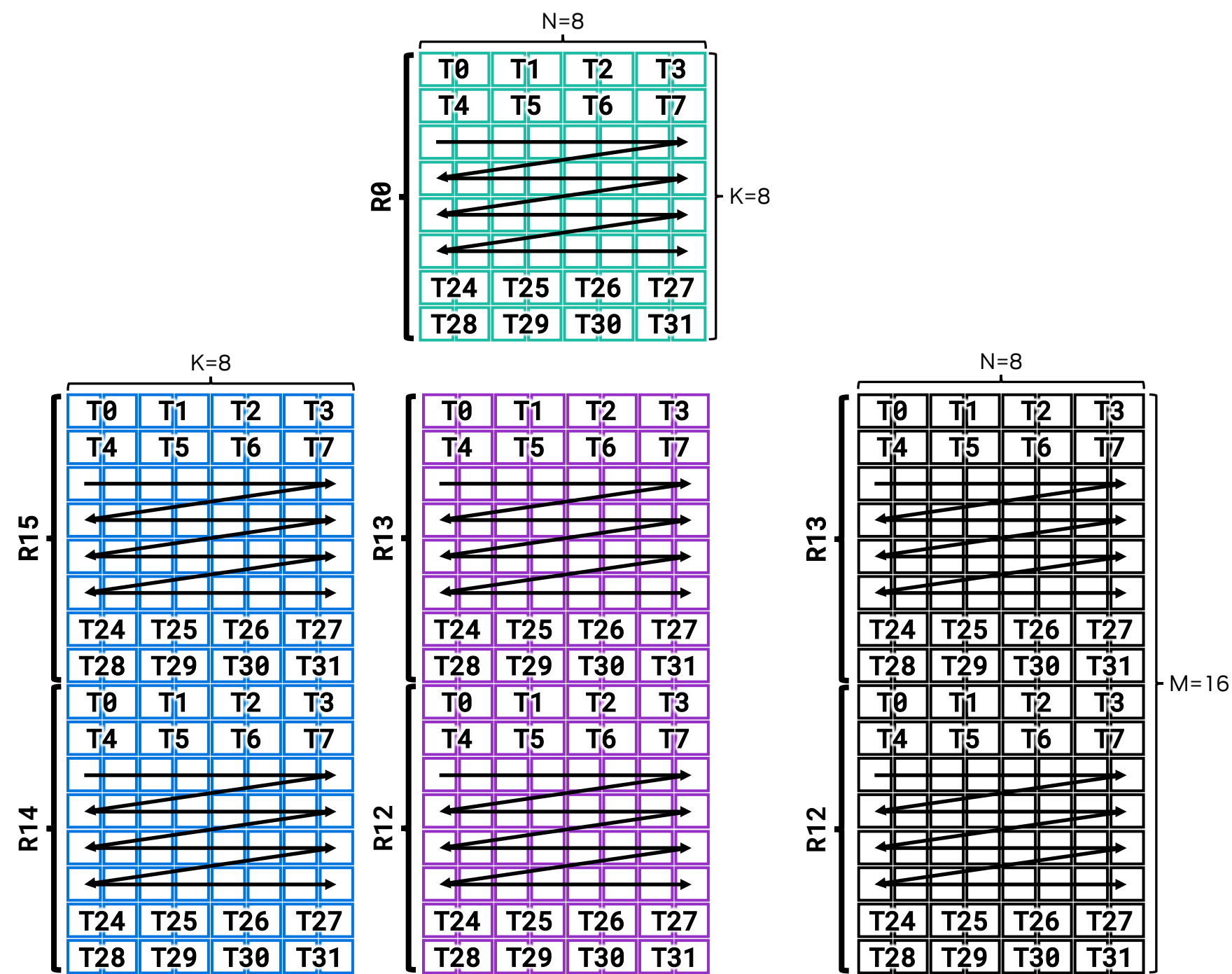
- All 32 threads in a warp execute combined MMA operation(s)
- Input/Output matrices held in registers across the warp
 - Layout of matrices in registers across warp defined by:
 - Size of MMA operation
 - Used formats
 - Register input/output size constrains overall MMA size
- Multiple MMA operations per instruction, if possible
 - Like HFMA executes *two* fp16 fused-multiple-adds with one instruction
 - $32 \text{ threads} \times 2 \times (1 \text{ multiplication} + 1 \text{ addition}) = 128 \text{ fp16 ops}$
 - Example: `mma.m8n8k4` for fp16 calculates *four* MMA operations
 - $1 \text{ warp} \times 4 \times (8 \times 8 \times 4 \text{ multiplications} + 8 \times 8 \times 4 \text{ additions}) = 2,048 \text{ fp16 ops}$
 - Applicability depends on format, MMA size, and architecture

Requirements:

- Data needs to be loaded in registers before MMA instruction
- Executed as single warp instruction
 - No partial warps, no partial predication allowed

Warp Level MMA Instructions

Example: Matrix Layout for 16x8x8 fp16 matrix multiply



SASS:

Assumes data is already loaded into registers
HMMA.1688.F16.F16 R12, R14, R0, R12;*

Matrix A:

- Size: 16x8 x 2 bytes = 256 bytes
- Requires two 32bit registers (32 threads x 2 x 4 bytes = 256 bytes)
- Stored in R14 and R15

Matrix B:

- Size: 8x8 x 2 bytes = 128 bytes
- Requires one 32bit register (32 threads x 1 x 4 bytes = 128 bytes)
- Stored in R0

Matrix C, D:

- Size: 16x8 x 2 bytes = 256 bytes
- Each requires two 32bit registers (32 threads x 2 x 4 bytes = 256 bytes)
- C is stored in R12 and R13; D is stored in R12 and R13 for accumulation

Summary for 1 warp (Comparison to HFMA):

- 7 32bit registers per thread (4)
- 1 warp instruction executed (1)
- 2,048 fp16 ops = 16 x 8 x 8 x (1 multiplication + 1 addition) (128)

* Instruction might differ on architectures earlier than Hopper/GH100

Warp Level MMA Instructions

Profiling Results: Matrix Layout for 16x8x8 fp16 matrix multiply

Register Dependency Analysis:

Input Register Dependencies				Output Register Dependencies	
Location	Instruction Category	Attributed Live Registers	Output Registers	Location	Instruction Category
Line 4 (0x1e01124830)	Miscellaneous	6	2	Line 26 (0x1e01124990)	Load/Store
Line 5 (0x1e01124840)	Miscellaneous	6	2	Line 27 (0x1e011249a0)	Load/Store
Line 9 (0x1e01124880)	Movement	6	1		

5 Input Registers 2 Output Registers

Source Page:

#	Label	Address	Source	Instructions Executed	Avg. Predicated-Op Threads Executed
19		0000001e 01124920	@!P0 LDG.E R14, desc[UR4][R6.64+0x8]	1	
20		0000001e 01124930	@!P0 LDG.E R15, desc[UR4][R8.64+0xc]	1	
> 21		0000001e 01124940	HMMMA.1688.F16 R12, R14, R0, R12	1	32
22		0000001e 01124950	@!UPT UIADD3 URZ, UPT, UPT, URZ, URZ, U	1	
23		0000001e 01124960	@!UPT UIADD3 URZ, UPT, UPT, URZ, URZ, U	1	

smsp__inst_executed_pipe_tensor_subpipe_hmma_op_hmma.sum

Tensor Core Roofline Table:

	# Operations	# Operations / Cycle	# Operations / s	Peak %	Peak Operations / Cycle	Peak Operations / s
Op:hmma Src:bf16 Dst:fp32 Sparsity:off	0	0	0	0	24,576	59,864,975,609,756.09
Op:hmma Src:bf16 Dst:fp32 Sparsity:on	0	0	0	0	49,152	119,729,951,219,512.19
Op:hmma Src:fp16 Dst:fp16 Sparsity:off	2,048	0.64	1,560,975,609.76	0.00	49,152	119,729,951,219,512.19
Op:hmma Src:fp16 Dst:fp16 Sparsity:on	0	0	0	0	98,304	239,459,902,439,024.38
Op:hmma Src:fp16 Dst:fp32 Sparsity:off	0	0	0	0	24,576	59,864,975,609,756.09

sm__ops_path_tensor_op_hmma_src_fp16_dst_fp16_sparsity_off.sum

SASS:

Assumes data is already loaded into registers
HMMMA.1688.F16.F16 R12, R14, R0, R12;

Matrix A:

- Size: 16x8 x 2 bytes = 256 bytes
- Requires two 32bit registers (32 threads x 2 x 4 bytes = 256 bytes)
- Stored in R14 and R15

Matrix B:

- Size: 8x8 x 2 bytes = 128 bytes
- Requires one 32bit register (32 threads x 1 x 4 bytes = 128 bytes)
- Stored in R0

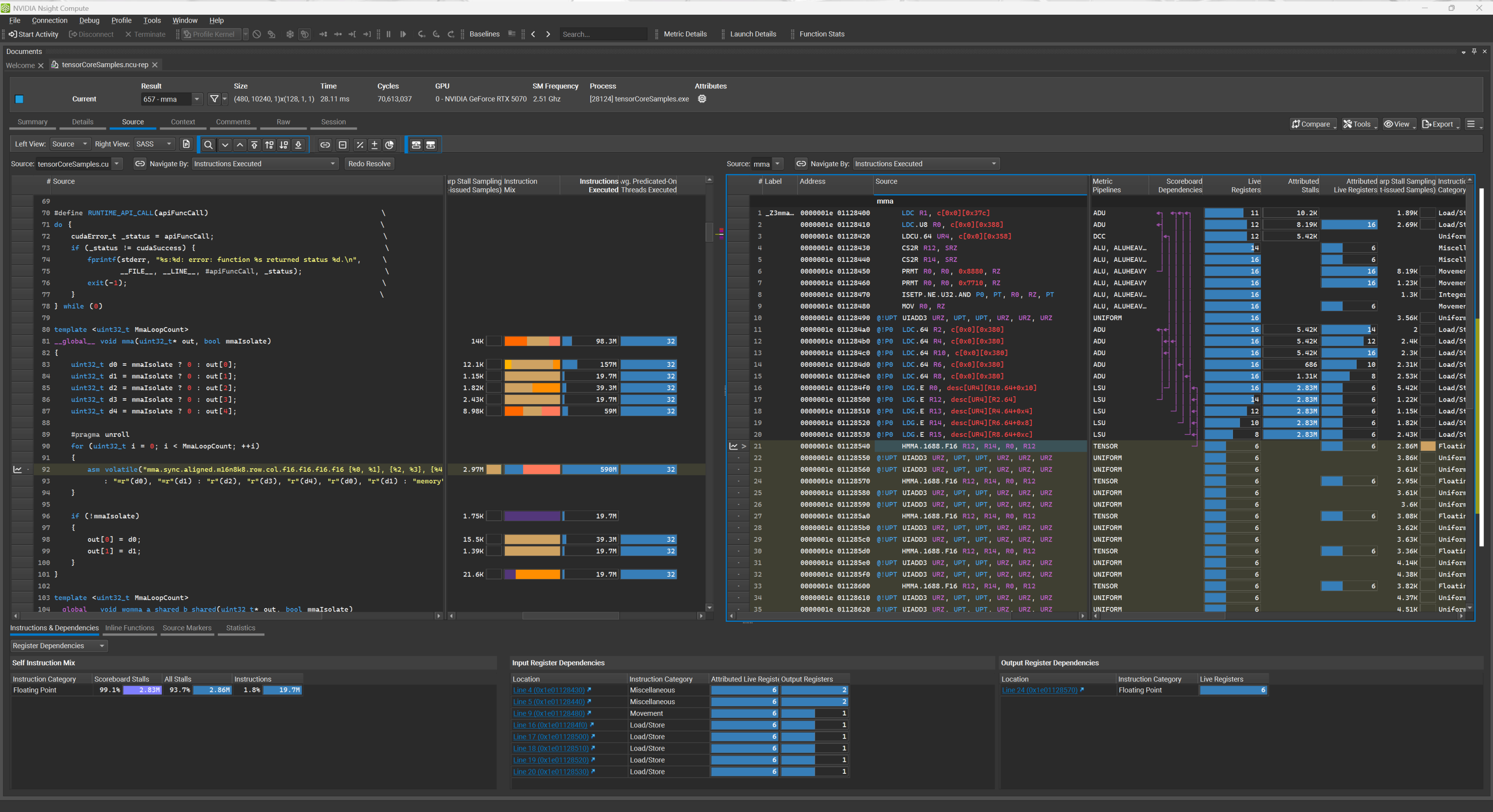
Matrix C, D:

- Size: 16x8 x 2 bytes = 256 bytes
- Each requires two 32bit registers (32 threads x 2 x 4 bytes = 256 bytes)
- C is stored in R8 and R9; D is stored in R12 and R13 for accumulation

Summary for 1 warp (Comparison to HFMA):

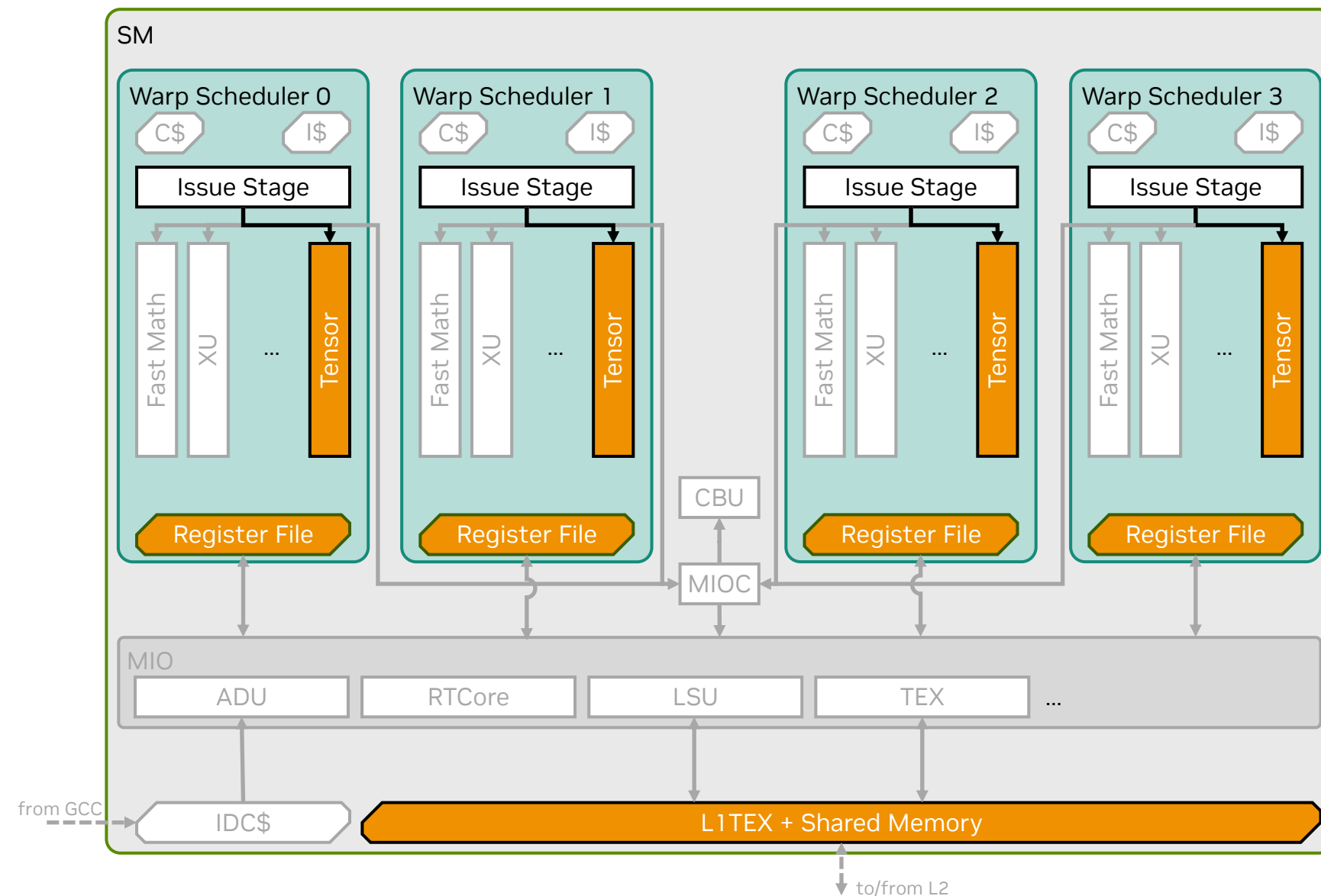
- 7 32bit registers per thread (4)
- 1 warp instruction executed (1)
- 2,048 fp16 ops = 16 x 8 x 8 x (1 multiplication + 1 addition) (128)

Warp Level MMA Instruction



Warpgroup Level MMA Instructions

Overview



Overall Idea:

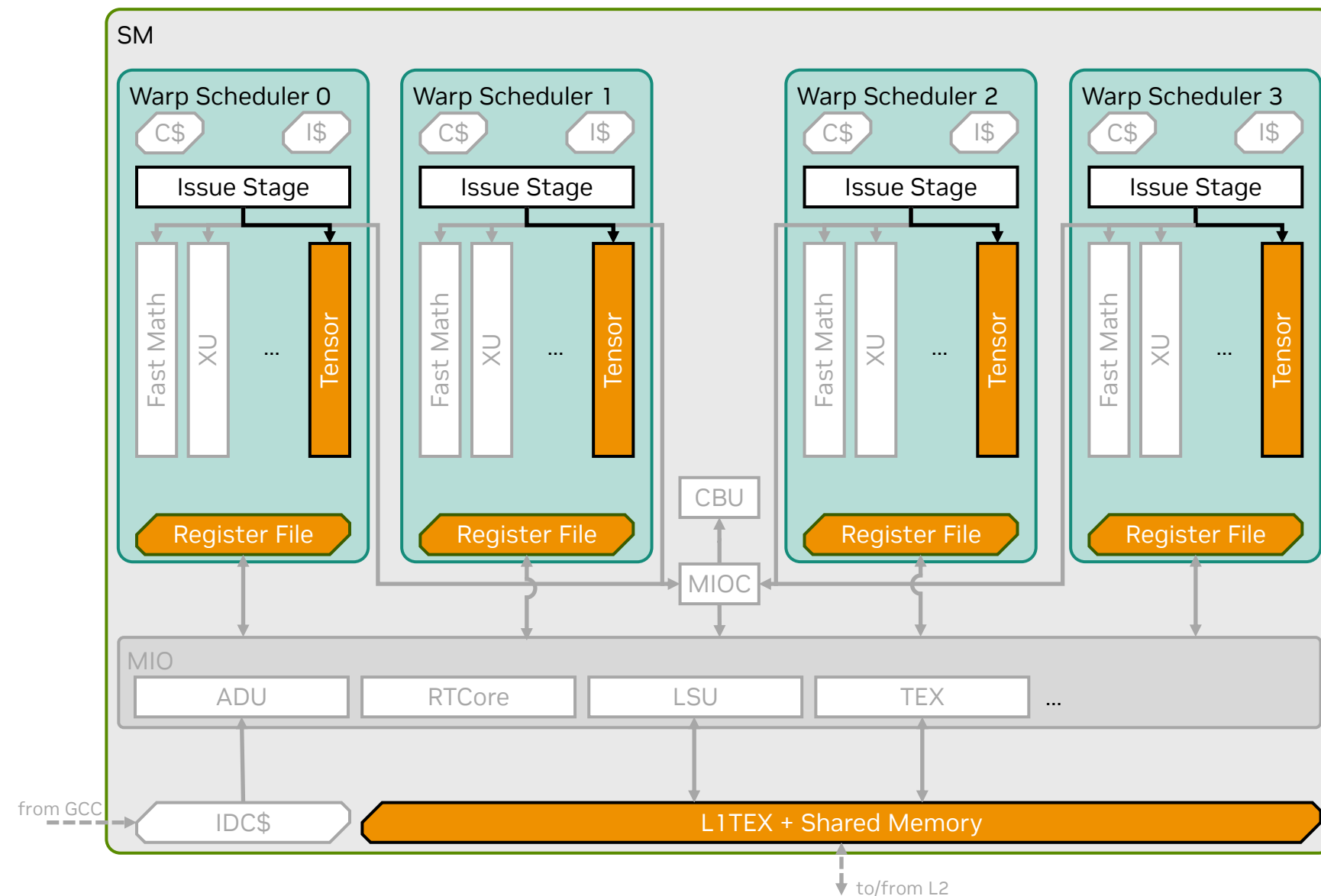
- Combine Tensor Cores of all 4 warp schedulers within an SM to:
 - Maximize data reuse (i.e. reduce overall data movement)
 - Increase matrix sizes
- Direct fetch of matrices A and B from shared memory
 - Broadcast to each warp scheduler
 - Avoids data movement through register file
 - Matrices C and D in SM's combined register file (increased space vs single warp scheduler)
- Asynchronous execution
 - Execution sequence: arrive -> wait -> complete

Requirements:

- Executed as warpgroup (4 warps within a block, 1 per warp scheduler within the same SM)
 - Requires block sizes of 4+ warps
 - If block size is more than 4, a valid group must be 4 consecutive warps with lowest warp_rank % 4 = 0
 - All 4 warps need to synchronize, and all execute the MMA instruction
 - New stall reason: warpgroup_arrive, gmma
 - No partial warps, no partial predication allowed

Warpgroup Level MMA Instructions

Code (64x32x16 fp16 matrix multiply)



Matrix: A
Dim: 64x16
Bytes: 2048

Matrix: B
Dim: 16x32
Bytes: 1024

SASS Sequence (non-pipelined):

```
# Load Matrix A
# Each thread loads 128 / 8 = 16 bytes
LDGSTS.E.128 [R0], [R1.U32]

# Load Matrix B
# Each thread loads 64 / 8 = 8 bytes
LDGSTS.E.64 [R2], [R3.U32]

# Completion of LDGSTS
LDGDEPBAR

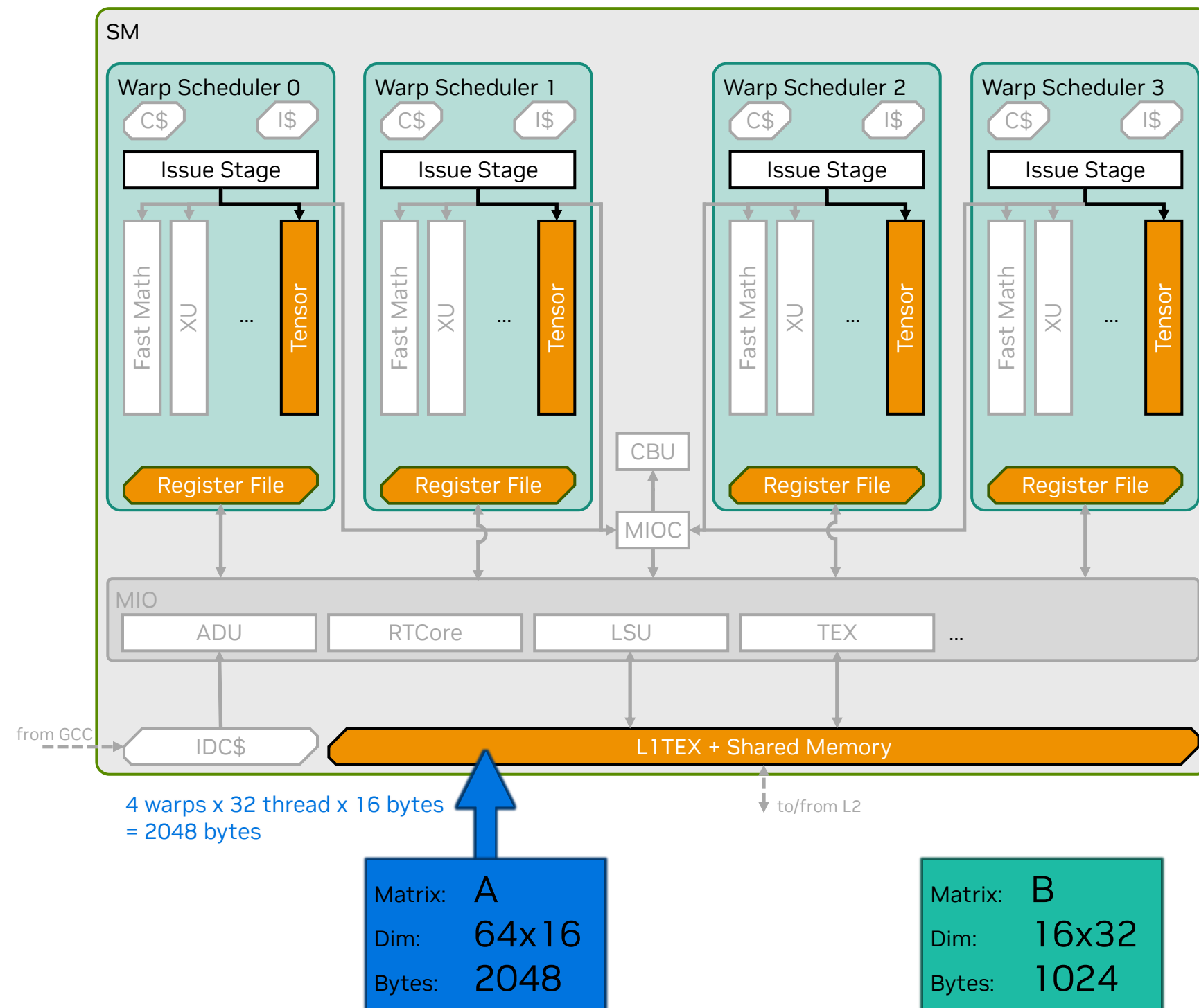
# Signal warp arrival at HGMMMA instruction sequence
WARPGROUP.ARRIVE

# Execute D=A*B across 4 warps (UP0=0)
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0

# Wait for HGMMMA operation to complete
WARPGROUP.DEPBAR.LE gsb0 0
```

Warpgroup Level MMA Instructions

Code (64x32x16 fp16 matrix multiply)



SASS Sequence (non-pipelined):

```
# Load Matrix A
# Each thread loads 128 / 8 = 16 bytes
LDGSTS.E.128 [R0], [R1.U32]

# Load Matrix B
# Each thread loads 64 / 8 = 8 bytes
LDGSTS.E.64 [R2], [R3.U32]

# Completion of LDGSTS
LDGDEPBAR

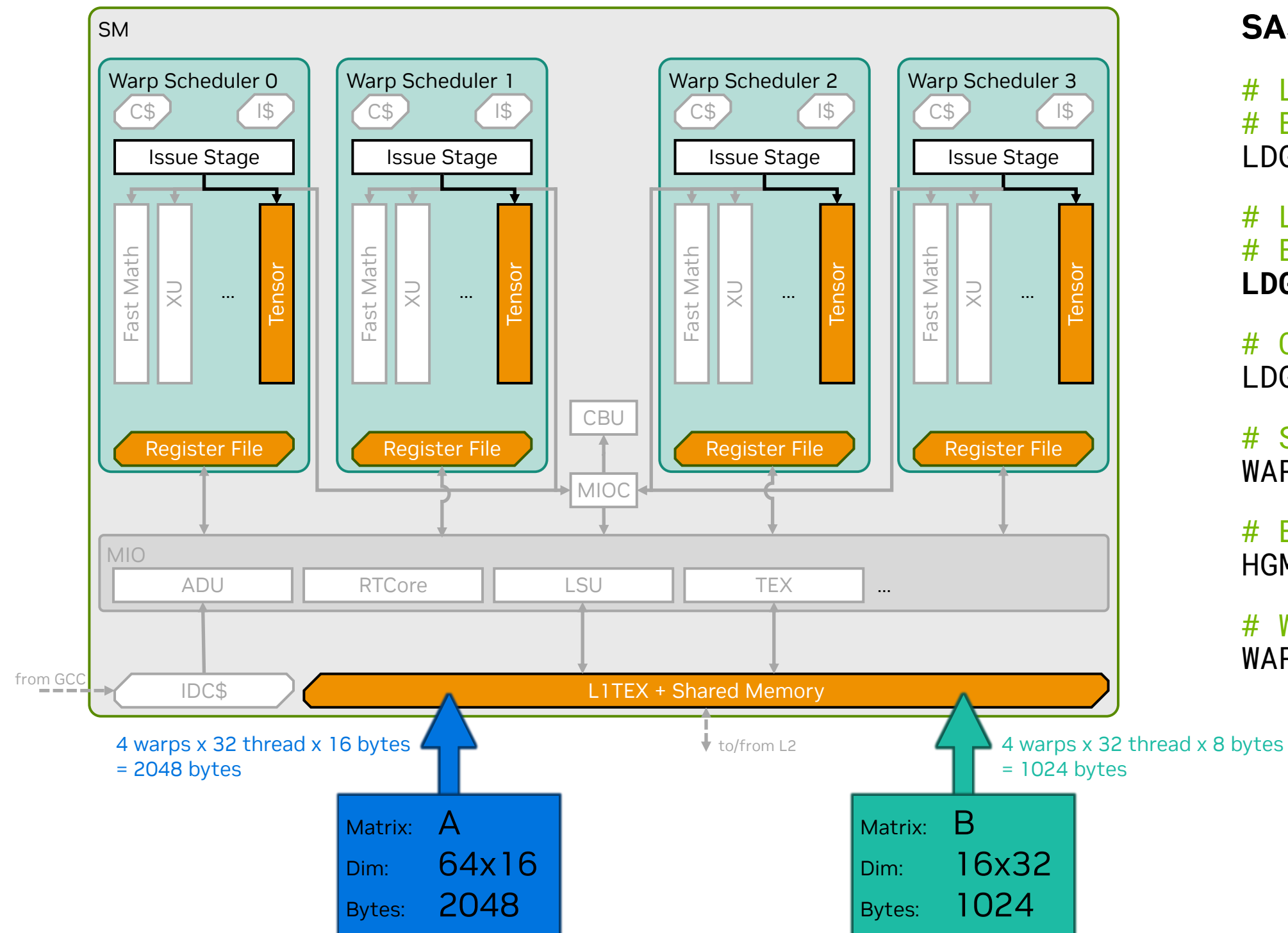
# Signal warp arrival at HGMMMA instruction sequence
WARPGROUP.ARRIVE

# Execute D=A*B across 4 warps (UP0=0)
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0

# Wait for HGMMMA operation to complete
WARPGROUP.DEPBAR.LE gsb0 0
```

Warpgroup Level MMA Instructions

Code (64x32x16 fp16 matrix multiply)



SASS Sequence (non-pipelined):

```
# Load Matrix A
# Each thread loads 128 / 8 = 16 bytes
LDGSTS.E.128 [R0], [R1.U32]

# Load Matrix B
# Each thread loads 64 / 8 = 8 bytes
LDGSTS.E.64 [R2], [R3.U32]

# Completion of LDGSTS
LDGDEPBAR

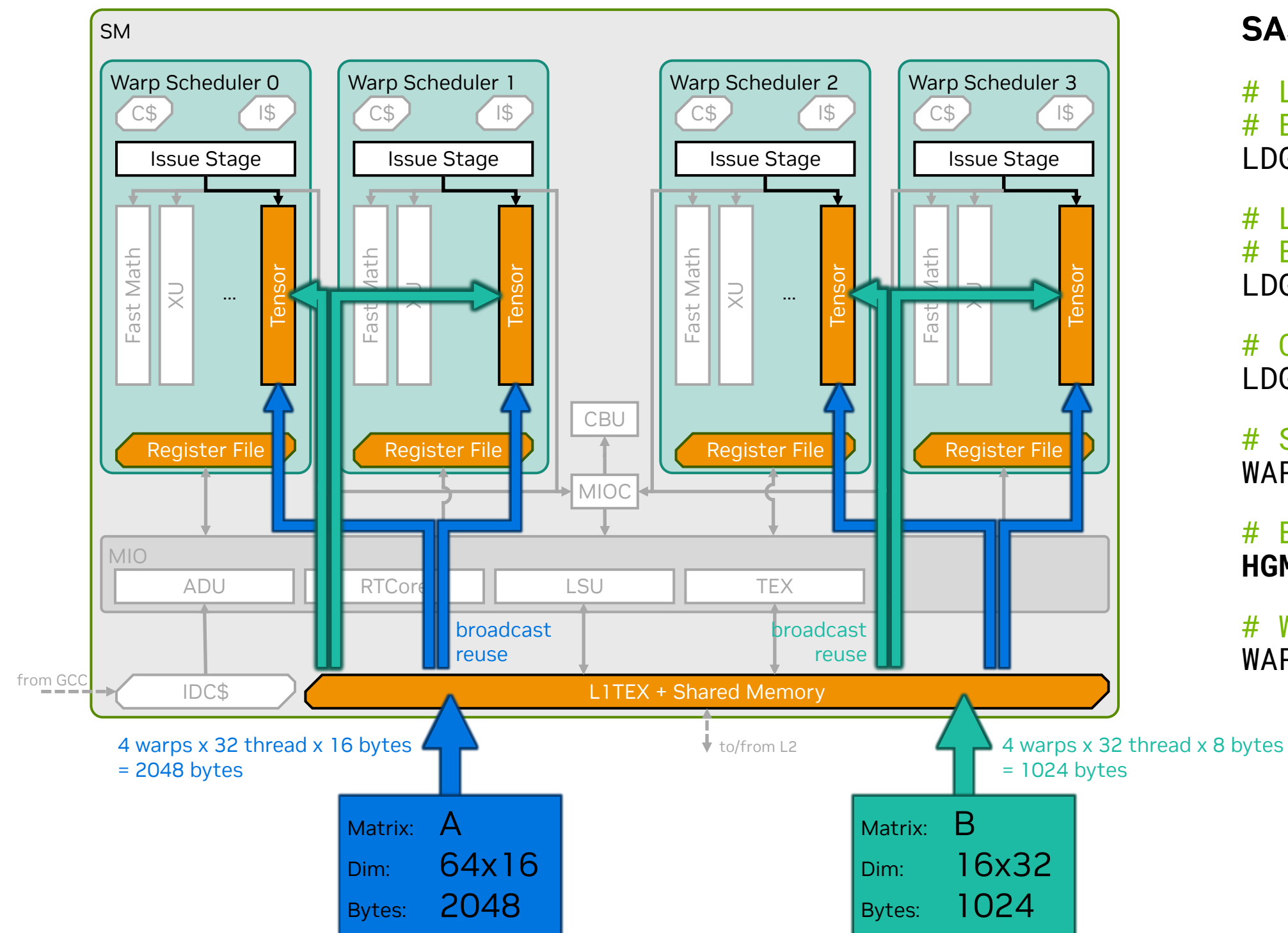
# Signal warp arrival at HGMMMA instruction sequence
WARPGROUP.ARRIVE

# Execute D=A*B across 4 warps (UP0=0)
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0

# Wait for HGMMMA operation to complete
WARPGROUP.DEPBAR.LE gsb0 0
```

Warpgroup Level MMA Instructions

Code (64x32x16 fp16 matrix multiply)



SASS Sequence (non-pipelined):

```
# Load Matrix A
# Each thread loads 128 / 8 = 16 bytes
LDGSTS.E.128 [R0], [R1.U32]

# Load Matrix B
# Each thread loads 64 / 8 = 8 bytes
LDGSTS.E.64 [R2], [R3.U32]

# Completion of LDGSTS
LDGDEPBAR

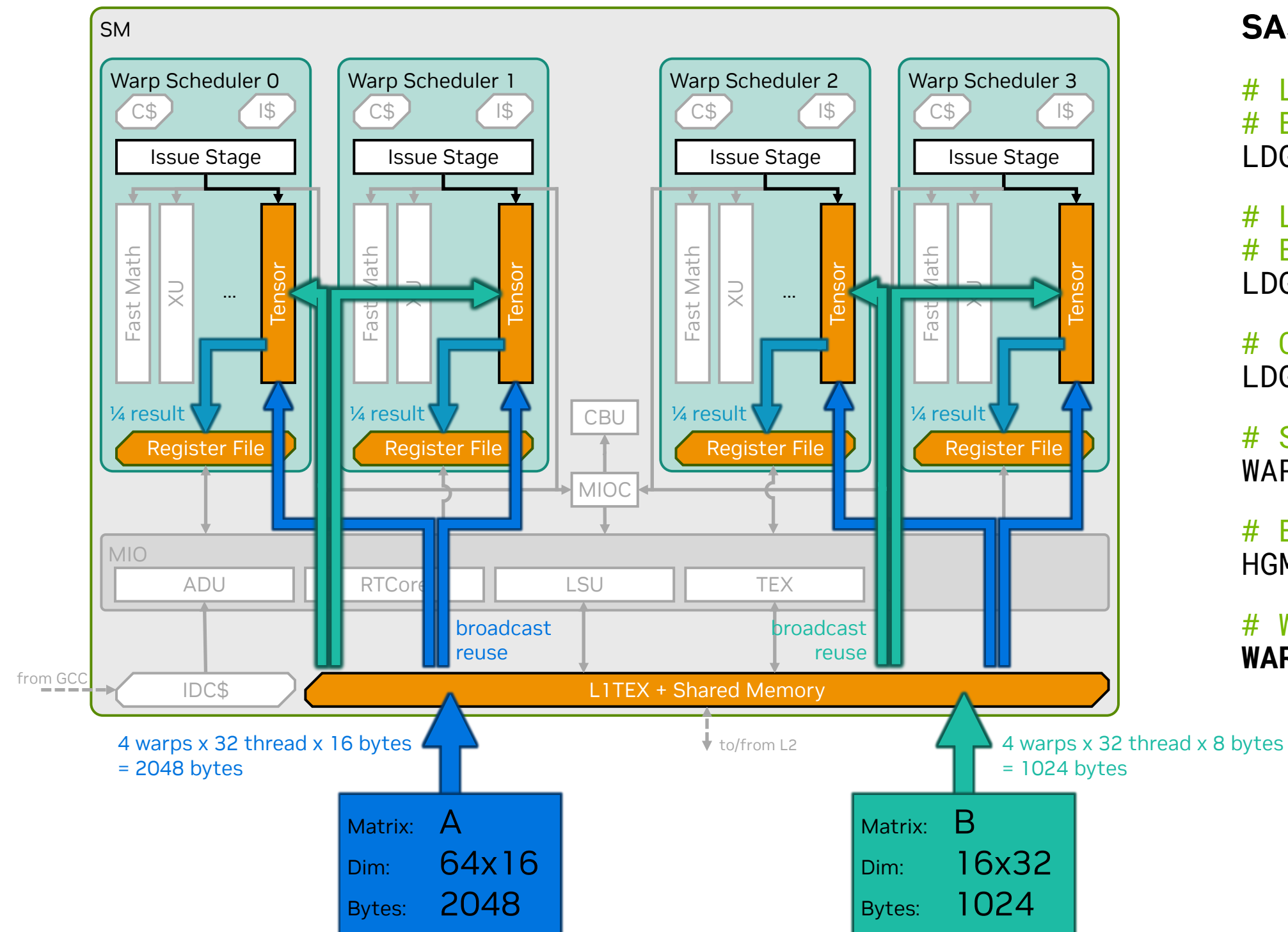
# Signal warp arrival at HGMMMA instruction sequence
WARPGROUP.ARRIVE

# Execute D=A*B across 4 warps (UP0=0)
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0

# Wait for HGMMMA operation to complete
WARPGROUP.DEPBAR.LE gsb0 0
```

Warpgroup Level MMA Instructions

Code (64x32x16 fp16 matrix multiply)



SASS Sequence (non-pipelined):

```
# Load Matrix A
# Each thread loads 128 / 8 = 16 bytes
LDGSTS.E.128 [R0], [R1.U32]

# Load Matrix B
# Each thread loads 64 / 8 = 8 bytes
LDGSTS.E.64 [R2], [R3.U32]

# Completion of LDGSTS
LDGDEPBAR

# Signal warp arrival at HGMMMA instruction sequence
WARPGROUP.ARRIVE

# Execute D=A*B across 4 warps (UP0=0)
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0

# Wait for HGMMMA operation to complete
WARPGROUP.DEPBAR.LE gsb0 0
```

Warpgroup Level MMA Instructions

Profiling Results: Matrix Layout for 64x16x16 fp16 with fp32 accumulate matrix multiply

Source Page:

#	Label	Address	Source	Instruction Category	Instructions Executed	Avg. Predicated-On Threads Executed
29		0000001c 019b57c0	USHF.R.U64 UR6, UR6, 0x4, UR6	Uniform Datapath	4	32
30		0000001c 019b57d0	IMAD.SHL.U32 R5, R5, 0x8, RZ	Integer	4	32
31		0000001c 019b57e0	ISETP.GT.U32.OR P0, PT, R5,	Integer	4	32
32		0000001c 019b57f0	HGMMMA.64x16x16.F32 R24, gdesc	Warpgroup	4	32
33		0000001c 019b5800	WARPGROUP.DEPBAR.LE gsb0, 0x0	Warpgroup	4	32

smsp__inst_executed_pipe_tensor_op_gmma.sum

Tensor Core Roofline Table:

	# Operations	# Operations / Cycle	# Operations / s	Peak %	Peak Operations / Cycle	Peak Operations / s
Op:bgmma Src:int1	0	0	0	0	7,471,104	13,265,165,473,684,208
Op:bmma Src:int1	0	0	0	0	4,980,736	8,843,443,649,122,806
Op:hgmma Src:bf16 Dst:fp32 Sparsity:off	0	0	0	0	466,944	829,072,842,105,263
Op:hgmma Src:bf16 Dst:fp32 Sparsity:on	0	0	0	0	933,888	1,658,145,684,210,526
Op:hgmma Src:fp16 Sparsity:off	32,768	7.59	13,473,684,210.53	0.00	466,944	829,072,842,105,263
Op:hgmma Src:fp16 Sparsity:on	0	0	0	0	933,888	1,658,145,684,210,526

sm__ops_path_tensor_op_hgmma_src_fp16_sparsity_off.sum

SASS:

Assumes data is already loaded
HGMMMA.64x16x16.F32 R24, gdesc[UR4], R24.gsb0;

Matrix A:

- Size: 64x16 x 2 bytes = 2048 bytes
- Stored in shared memory of the SM executing the warpgroup
- Descriptor specifies location

Matrix B:

- Size: 16x16 x 2 bytes = 512 bytes
- Stored in shared memory of the SM executing the warpgroup
- Descriptor specifies location

Matrix C, D:

- Size: 64x16 x 4 bytes = 4096 bytes
- Each requires eight 32bit registers (32 threads x 8 x 4 bytes x 4 warps = 4096 bytes)
- C, D are the same metric for accumulation. Stored in R24-R31

Summary for 1 warpgroup (4 warps):

- 8 32bit registers per thread
- 4 warp instruction executed
- 32,768 ops = 64 x 16 x (16 multiplications + 15 additions) in fp16 + 64 x 16 additions in fp32

Pipelining with Tensor Memory Accelerator (TMA)

Problem Statement

Load/Prolog:

- Predominately address calculation for global and shared memory addresses
- Takes SM resources away from 'actual' MMA work
- Error prone (many packing layouts, translations, swizzling, ...)

SASS Sequence:

```
# Load Matrix A
# Each thread loads 128 / 8 = 16 bytes
LDGSTS.E.128 [R0], [R1.U32]

# Load Matrix B
# Each thread loads 64 / 8 = 8 bytes
LDGSTS.E.64 [R2], [R3.U32]

# Completion of LDGSTS
LDGDEPBAR

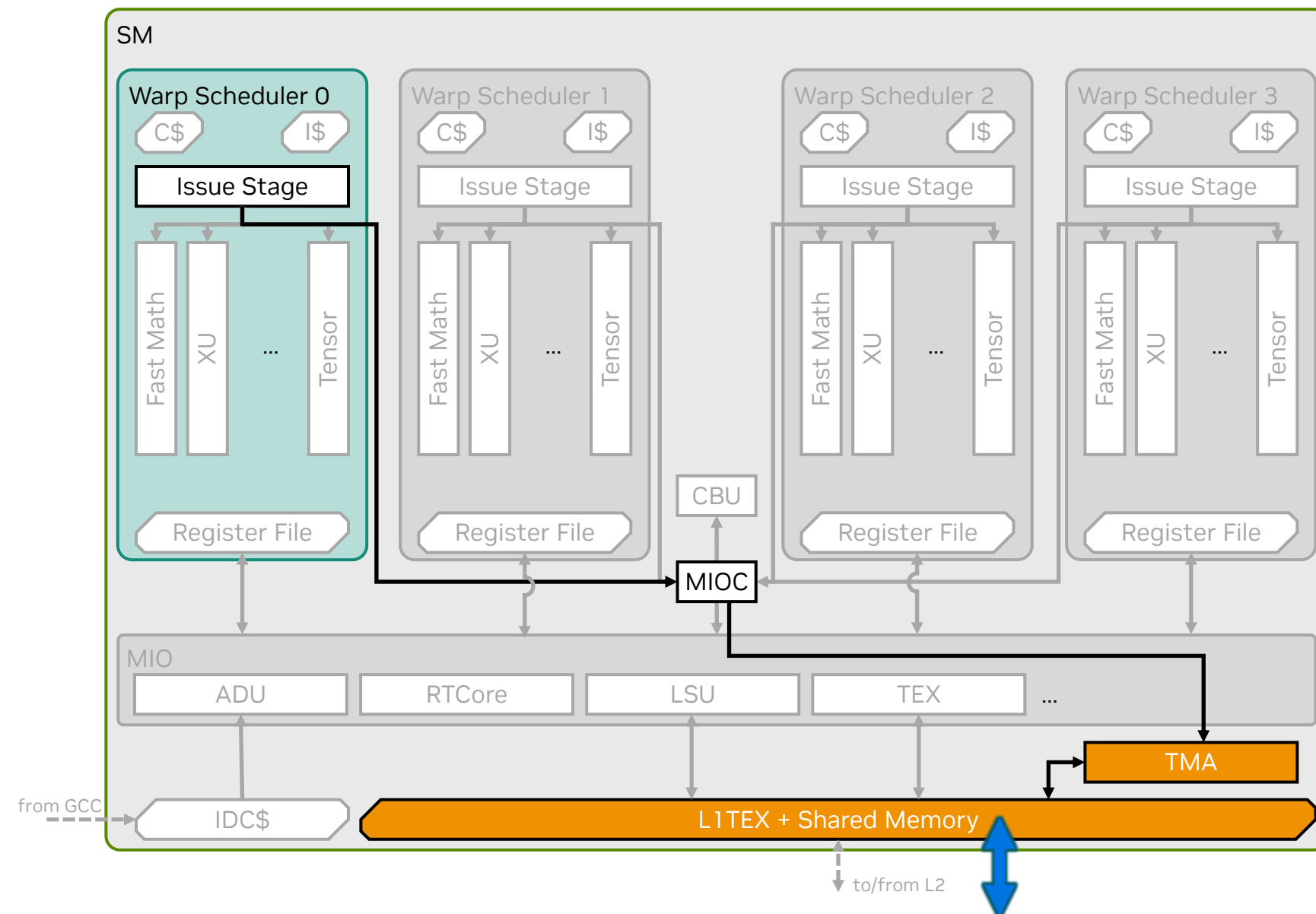
# Signal warp arrival at HGMMMA instruction sequence
WARPGROUP.ARRIVE

# Execute D=A*B across 4 warps (UP0=0)
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0

# Wait for HGMMMA operation to complete
WARPGROUP.DEPBAR.LE gsb0 0
```

Pipelining with Tensor Memory Accelerator (TMA)

Overview

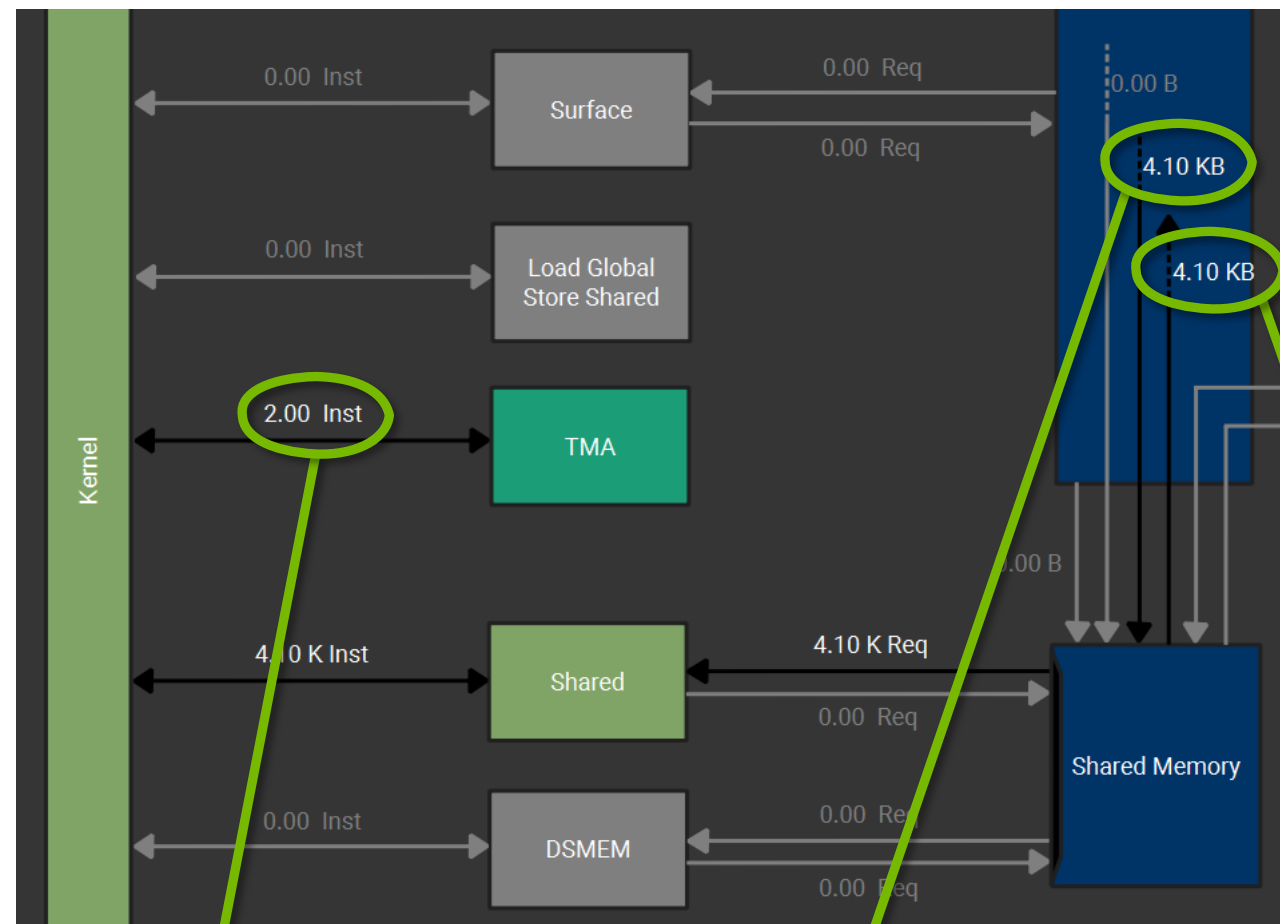


Overall Idea:

- Dedicated unit to move data between global and shared memory
 - Copies are defined by memory/tensor descriptors
 - Supports global to shared and shared to global memory
 - Removes costly calculation of memory addresses (up to 5d math)
 - Enables easier definition of transformations (transposes, swizzles, ...)
 - Supports broadcast from global memory to shared memory of all/some blocks within a cluster
 - Memory bandwidth amplification, i.e. less overall global memory reads
 - Asynchronous execution
 - Synchronization and completion tracking through shared memory barriers
- Single-threaded
 - One thread can prepare and issue TMA copy (UTMALDG, UTMASG, ...)
 - Decouples required instructions from copied data size, format, transformations
 - Single TMA copy can fill all of shared memory
- Introduced in Hopper/GH100, supported in all newer chips

Pipelining with Tensor Memory Accelerator (TMA)

Profiling Results



Overall Idea:

- Dedicated unit to move data between global and shared memory
 - Copies are defined by memory/tensor descriptors
 - Supports global to shared and shared to global memory
 - Removes costly calculation of memory addresses (up to 5d math)
 - Enables easier definition of transformations (transposes, swizzles, ...)
 - Supports broadcast from global memory to shared memory of all/some blocks within a cluster
 - Memory bandwidth amplification, i.e. less overall global memory reads
 - Asynchronous execution
 - Synchronization and completion tracking through shared memory barriers
- Single-threaded
 - One thread can prepare and issue TMA copy (UTMALDG, UTMASGT, ...)
 - Decouples required instructions from copied data size, format, transformations
 - Single TMA copy can fill all of shared memory
- Introduced in Hopper/GH100, supported in all newer chips

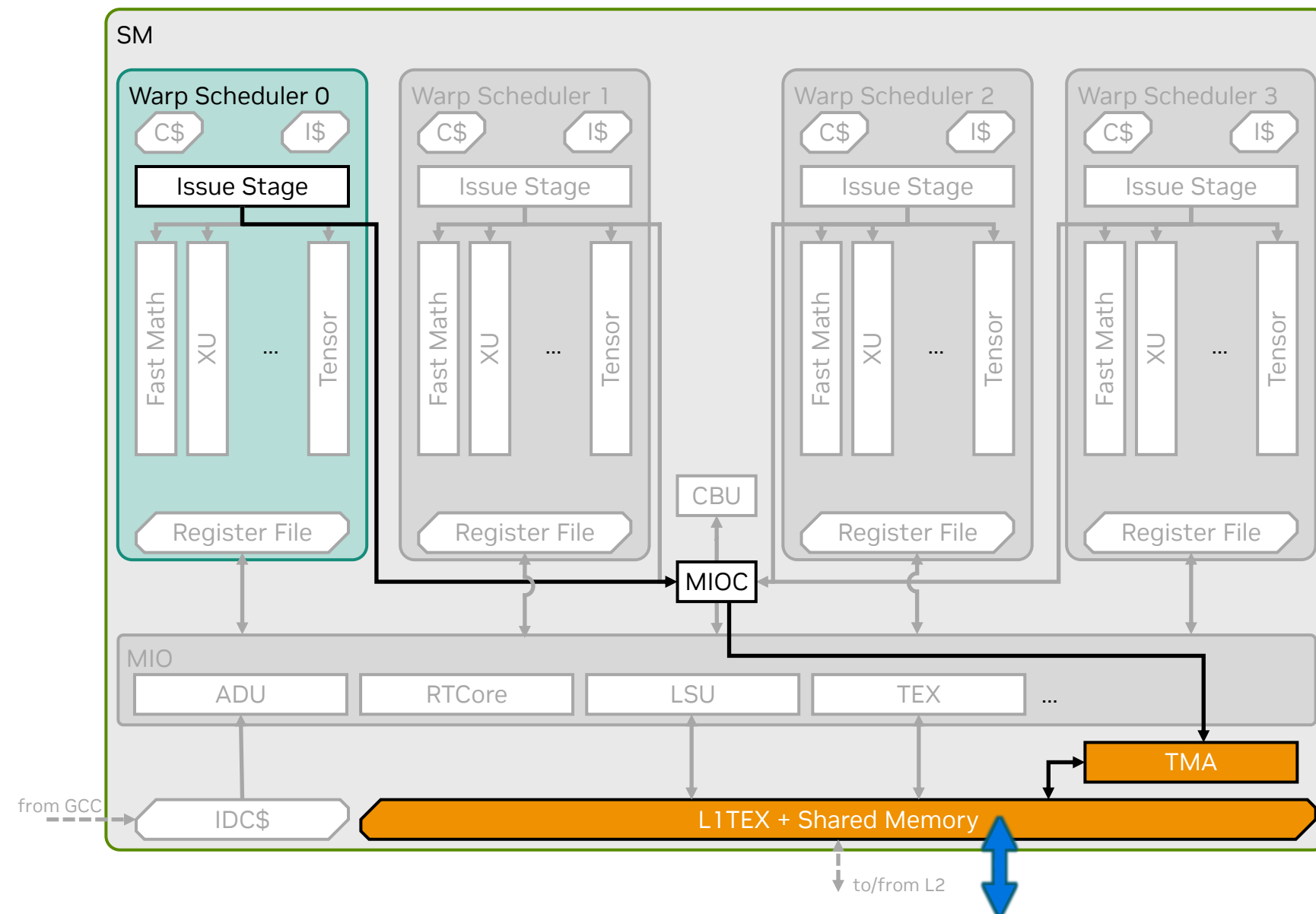
`smsp__inst_executed_op_tma_ld.sum`
`+ smsp__inst_executed_op_tma_st.sum`

`11tex__m_xbar211tex_read_sectors_mem_global_op_tma_ld.sum * 32`

`11tex__m_xbar211tex_read_sectors_mem_global_op_tma_st.sum * 32`

Pipelining with Tensor Memory Accelerator (TMA)

Example



SASS Sequence (simple pipeline):

Load Matrix A/B for Phase 1

```
UTMALDG.2D [UR8], [UR6]
UTMALDG.2D [UR10], [UR4]
```

Wait for TMA for Phase 1
BSYNC

Issue MMAs for Phase 1

```
WARGROUP.ARRIVE
HGMMMA.64x32x16.F16.F16 R0, gdesc[UR0], R16, UP0, gsb0
```

Load Matrix A/B for Phase 2

```
UTMALDG.2D [UR4], [UR2]
UTMALDG.2D [UR8], [UR6]
```

Wait for TMA for Phase 2
BSYNC

Issue MMAs for Phase 2

```
WARGROUP.ARRIVE
HGMMMA.64x32x16.F16.F16 R4, gdesc[UR10], R20, UP0, gsb0
```

Wait For Phase 1 MMAs to complete
WARGROUP.DEPBAR.LE gsb0 1

Continue with next phase

...

Warpgroup Level MMA Instructions

FileConnectionDebugProfileToolsWindowHelp

Start ActivityDisconnectTerminateProfile Kernel

Documents

Welcome X tensorCoreSamples_gh100.ncu-rep X

ResultSizeTimeCyclesGPUSM FrequencyProcessAttributes

Current642 - wgmma_a_shared_b_shared(1, 1, 1)x(128, 1, 1)2.43 us4,1930 - NVIDIA H100 PCIe1.70 Ghz[15292] tensorCoreSamples.exe

SummaryDetailsSourceContextCommentsRawSession

Left View: SourceRight View: SASS

Source: tensorCoreSamples.cuNavigate By: Instructions ExecutedRedo Resolve

Source

Attributed Stalls

Attributed arp Stall Sampling Instruction Live Registers Issued Samples Mix

Instructions Executed

```
117 float d3 = initValue;
118 float d4 = initValue;
119 float d5 = initValue;
120 float d6 = initValue;
121 float d7 = initValue;
122
123 asm volatile("wgmma.fence.sync.aligned;" ::: "memory");
124
125 #pragma unroll
126 for (uint32_t i = 0; i < MmaLoopCount; ++i)
127 {
128     asm volatile(
129         "wgmma.mma_async.sync.aligned.m64n16k16.f32.f16.f16 [%0, %1, %2, %3, %4, %5, %6, %7],\n"
130         : "+f"(d0), "+f"(d1), "+f"(d2), "+f"(d3), "+f"(d4), "+f"(d5), "+f"(d6), "+f"(d7)";
131     }
132
133 asm volatile("wgmma.commit_group.sync.aligned;" ::: "memory");
134 asm volatile("wgmma.wait_group.sync.aligned 0;" ::: "memory");
135
136 const uint32_t threadOffset = threadIdx.x * 8;
137 if (!mmaIsolate && threadOffset < (64 * 16 / 4))
138 {
139     out[0 + threadOffset] = __float_as_uint(d0);
140     out[1 + threadOffset] = __float_as_uint(d1);
141     out[2 + threadOffset] = __float_as_uint(d2);
142     out[3 + threadOffset] = __float_as_uint(d3);
143     out[4 + threadOffset] = __float_as_uint(d4);
144     out[5 + threadOffset] = __float_as_uint(d5);
145     out[6 + threadOffset] = __float_as_uint(d6);
146     out[7 + threadOffset] = __float_as_uint(d7);
147 }
148 else
149     (void)out;
150 (void)mmaIsolate;
151#endif
```

Source: wgmma_a_shared_b_shared

Navigate By: Instructions Executed

# Label	Address	Source	Instruction Category	Instructions Executed	Avg. Predicated-On Threads Executed	Address Space	Access Operation
12	0000001c 019b56b0	IMAD.MOV.U32 R26, RZ, RZ, R24	Integer	4	32		
13	0000001c 019b56c0	MOV R27, R24	Movement	4	32		
14	0000001c 019b56d0	IMAD.MOV.U32 R28, RZ, RZ, R24	Integer	4	32		
15	0000001c 019b56e0	MOV R29, R24	Movement	4	32		
16	0000001c 019b56f0	IMAD.MOV.U32 R30, RZ, RZ, R24	Integer	4	32		
17	0000001c 019b5700	MOV R31, R24	Movement	4	32		
18	0000001c 019b5710	S2UR UR6, SR_SWINHI	Uniform Datapath	4	32		
19	0000001c 019b5720	UMOV UR7, URZ	Uniform Datapath	4	32		
20	0000001c 019b5730	S2R R5, SR_TID.X	Miscellaneous	4	32		
21	0000001c 019b5740	WARPGROUP.ARRIVE	Warpgroup	4	32		
22	0000001c 019b5750	ULEA UR4, UR4, 0x400, 0x18	Uniform Datapath	4	32		
23	0000001c 019b5760	UMOV UR5, UR6	Uniform Datapath	4	32		
24	0000001c 019b5770	UMOV UR5, UR6	Uniform Datapath	4	32		
25	0000001c 019b5780	UIADD3 UR6, UR4, 0x800, URZ	Uniform Datapath	4	32		
26	0000001c 019b5790	ULOP3.LUT UR4, UR4, 0x3fff0, URZ, 0xc0, !UPT	Uniform Datapath	4	32		
27	0000001c 019b57a0	ULOP3.LUT UR6, UR6, 0x3fff0, URZ, 0xc0, !UPT	Uniform Datapath	4	32		
28	0000001c 019b57b0	USHF.R.U64 UR4, UR4, 0x4, URZ	Uniform Datapath	4	32		
29	0000001c 019b57c0	USHF.R.U64 UR6, UR6, 0x4, URZ	Uniform Datapath	4	32		
30	0000001c 019b57d0	IMAD.SHL.U32 R5, R5, 0x8, RZ	Integer	4	32		
31	0000001c 019b57e0	ISETP.GT.U32.OR P0, PT, R5, 0xff, P0	Integer	4	32		
> 32	0000001c 019b57f0	HGMMA.64x16x16.F32 R24, gdesc[UR4], R24.gsb0	Warpgroup	4	32	Shared	Load
33	0000001c 019b5800	WARPGROUP.DEPBAR.LE gsb0, 0x0	Warpgroup	4	32		
34	0000001c 019b5810 @P0	EXIT	Control	4	32		
35	0000001c 019b5820	LDC.64 R2, c[0x0][0x210]	Load/Store				
36	0000001c 019b5830	IMAD.WIDE.U32 R2, R5, 0x4, R2	Integer				
37	0000001c 019b5840	STG.E desc[UR8][R2.64], R24	Load/Store				
38	0000001c 019b5850	STG.E desc[UR8][R2.64+0x4], R25	Load/Store				
39	0000001c 019b5860	STG.E desc[UR8][R2.64+0x8], R26	Load/Store				
40	0000001c 019b5870	STG.E desc[UR8][R2.64+0xc], R27	Load/Store				
41	0000001c 019b5880	STG.E desc[UR8][R2.64+0x10], R28	Load/Store				
42	0000001c 019b5890	STG.E desc[UR8][R2.64+0x14], R29	Load/Store				
43	0000001c 019b58a0	STG.E desc[UR8][R2.64+0x18], R30	Load/Store				
44	0000001c 019b58b0	STG.E desc[UR8][R2.64+0x1c], R31	Load/Store				
45	0000001c 019b58c0	EXIT	Control				
46 .L_x_1	0000001c 019b58d0	BRA %.L_x_1 0x1c019b58d0	Control				

Instructions & DependenciesInline FunctionsSource MarkersStatistics

Register Dependencies

Self Instruction Mix

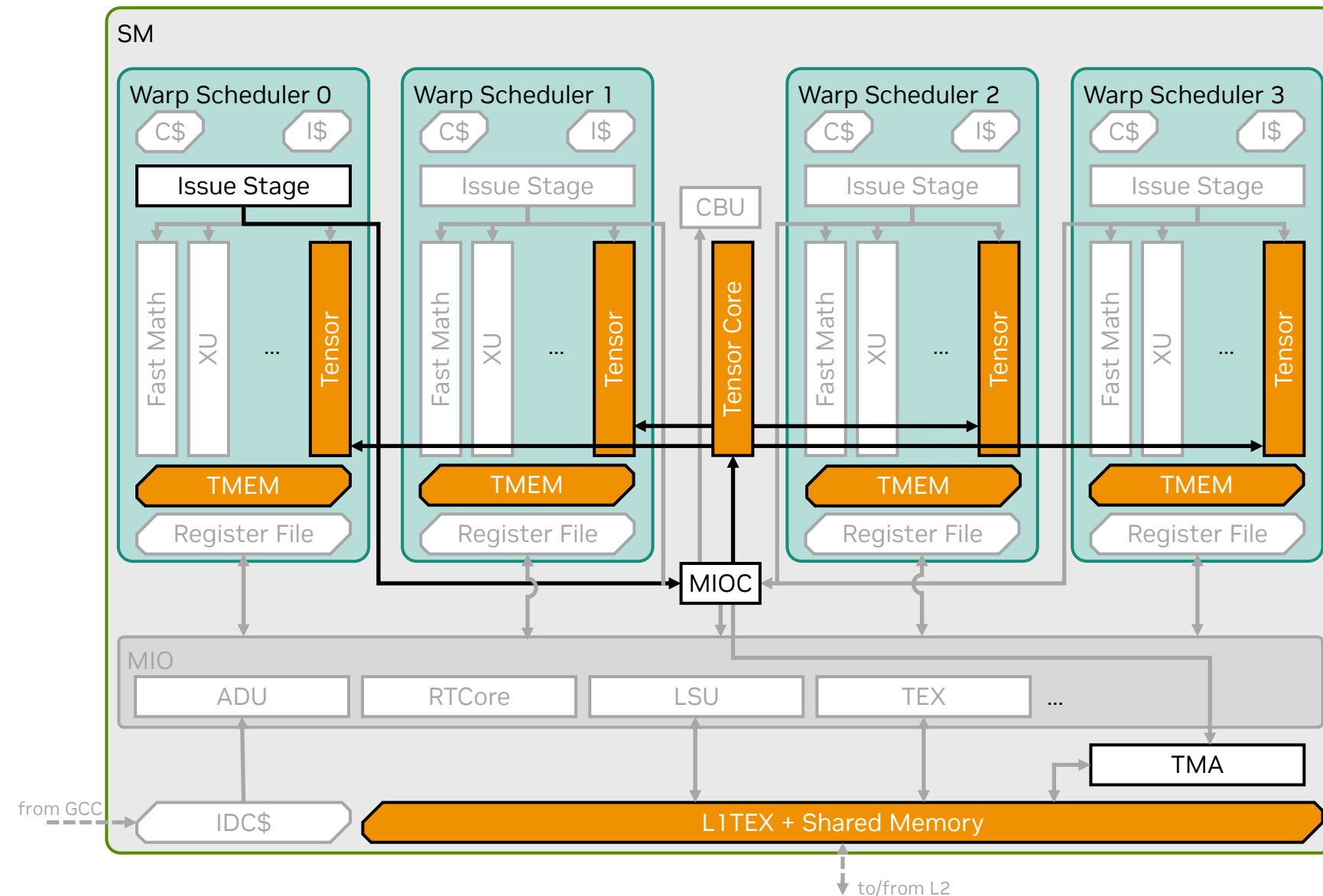
Instruction Category	Scoreboard Stalls	All Stalls	Instructions
Warpgroup			3.1% 4

Input Register Dependencies

Location	Instruction Category	Attributed Live Regist	Output Registers
Line 5 (0x1c019b5640)	Integer	10	1
Line 9 (0x1c019b5680)	Load/Store	10	1
Line 11 (0x1c019b56a0)	Movement	10	1
Line 12 (			

TensorCore 5th Generation Family Instructions

Overview



Overall Idea:

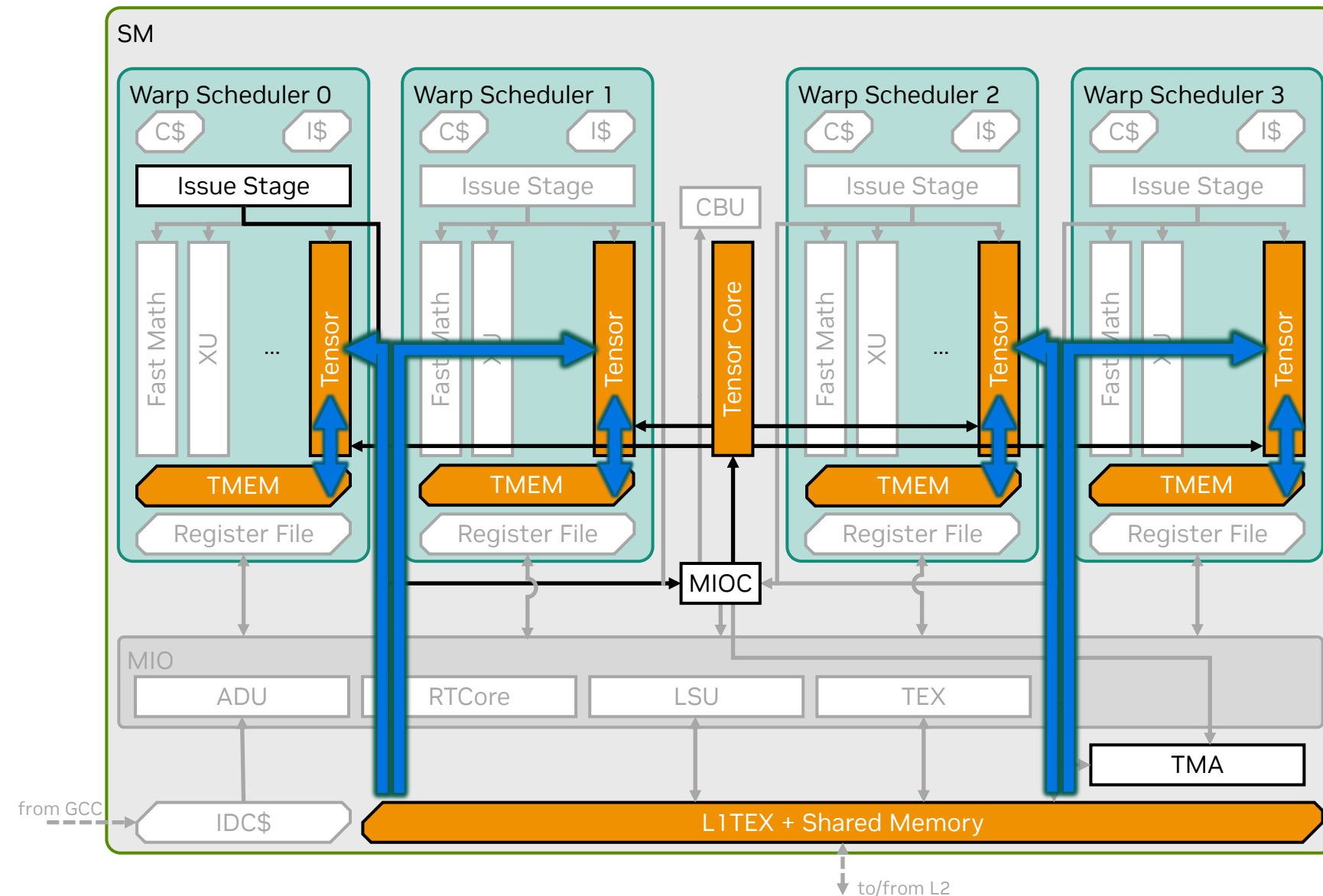
- New shared **Tensor Core** unit to control Tensor Pipelines
- tcgen05.mma/UTCMMA submitted by single thread
- Dedicated TensorCore Memory (**TMEM**)
 - Extra 64KB per warp scheduler
 - Explicit allocation/deallocation
 - Load/Store instructions from/to registers or shared memory
 - Loading from TMEM to RF is a per warp per warp scheduler operation
 - PTX exposure requires a operating to be executed by the warp group(s)
- Operand Locations (no more register file storage):
 - Matrix A from shared memory or TMEM
 - Matrix B from shared memory
 - Matrix D in TMEM
- Asynchronous execution
 - Designed with pipelining operations in mind

Requirements:

- Executed as 1 warpgroup (1 SM) or 2 warpgroups (on 2 SMs)
 - Requires block sizes of 4+ warps
 - If block size is more than 4, a valid group must be 4 consecutive warps with lowest warp_rank % 4 = 0

TensorCore 5th Generation Family Instructions

Overview



Overall Idea:

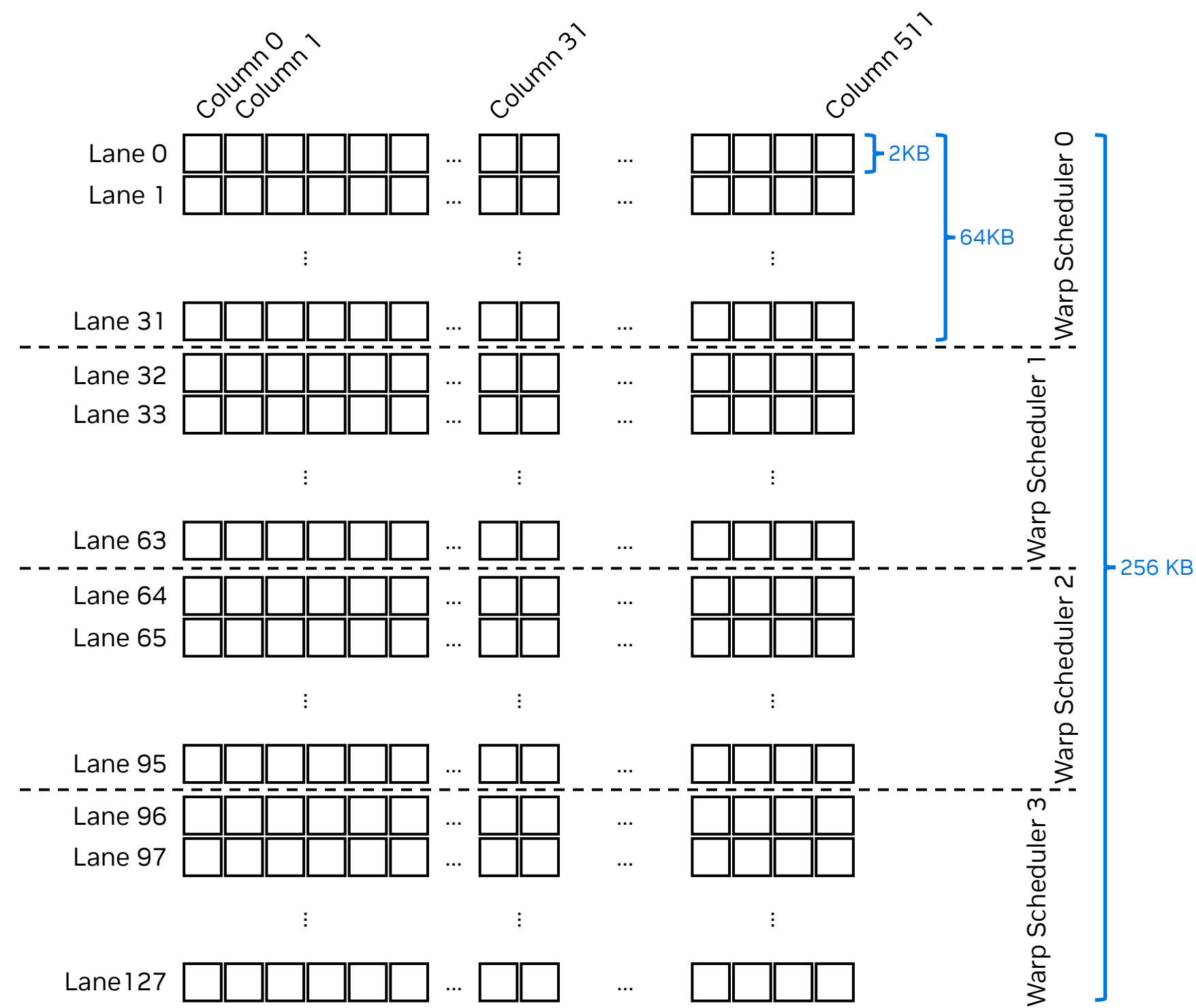
- New shared **Tensor Core** unit to control Tensor Pipelines
- tcgen05.mma/UTCMMA submitted by single thread
- Dedicated TensorCore Memory (**TMEM**)
 - Extra 64KB per warp scheduler
 - Explicit allocation/deallocation
 - Load/Store instructions from/to registers or shared memory
 - Loading from TMEM to RF is a per warp per warp scheduler operation
 - PTX exposure requires a operation to be executed by the warp group(s)
- Operand Locations (no more register file storage):
 - Matrix A from shared memory or TMEM
 - Matrix B from shared memory
 - Matrix D in TMEM
- Asynchronous execution
 - Designed with pipelining operations in mind

Requirements:

- Executed as 1 warpgroup (1 SM) or 2 warpgroups (on 2 SMs)
 - Requires block sizes of 4+ warps
 - If block size is more than 4, a valid group must be 4 consecutive warps with lowest warp_rank % 4 = 0

TensorCore 5th Generation Family Instructions

Tensor Core Memory



Layout:

- Matrix of 128 rows (lanes) x 512 columns of 4byte cells
 - Total size of 256KB, split across the 4 warp schedulers in row dimension
 - Allocation performed in terms of number of columns
 - Always allocates all 128 lanes per column
 - Allocation granularity of 32 columns
 - Addressing via 32bit values
 - [15 .. 0]: Column Index and [31 .. 16]: Lane Index

Allocation/Deallocation:

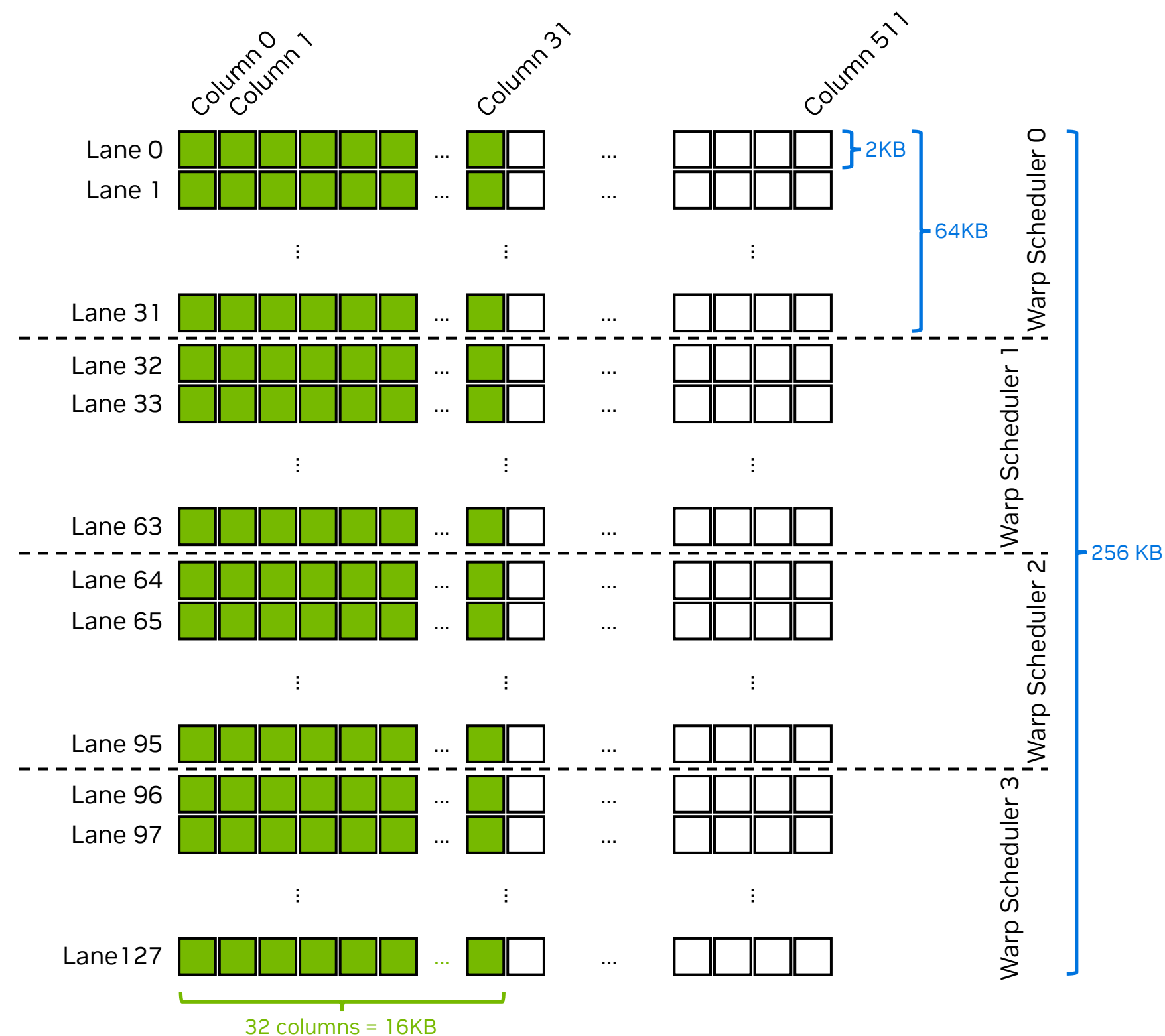
- Programmatic management by 1 warp within warpgroup (2 warps in Cluster)
 - tcgen05.alloc, tcgen05.dealloc, and tcgen05.relinquish_alloc_permit
 - Memory management instructions are blocking
 - Relinquish to signal all allocations are done, enables subsequent blocks to launch or allocate
- Must explicitly deallocate before kernel ends

Data Movements:

- From/To register file (tcgen05.ld/LDT[M] and tcgen05.st/STT[M])
 - Issued by all threads in warpgroup(s), each warp can only access its local TMEM lanes
- From shared memory (tcgen05.cp/UTCCP)
 - Issued by one thread in the warpgroup(s), equivalent to mma

TensorCore 5th Generation Family Instructions

Tensor Core Memory



Layout:

- Matrix of 128 rows (lanes) x 512 columns of 4byte cells
 - Total size of 256KB, split across the 4 warp schedulers in row dimension
 - Allocation performed in terms of number of columns
 - Always allocates all 128 lanes per column
 - Allocation granularity of 32 columns
 - Addressing via 32bit values
 - [15 .. 0]: Column Index and [31 .. 16]: Lane Index

Allocation/Deallocation:

- Programmatic management by 1 warp within warpgroup (2 warps in Cluster)
 - `tcgen05.alloc`, `tcgen05.dealloc`, and `tcgen05.relinquish_alloc_permit`
 - Memory management instructions are blocking
 - Relinquish to signal all allocations are done, enables subsequent blocks to launch or allocate
- Must explicitly deallocate before kernel ends

Data Movements:

- From/To register file (`tcgen05.ld/LDT[M]` and `tcgen05.st/STT[M]`)
 - Issued by all threads in warpgroup(s), each warp can only access its local TMEM lanes
- From shared memory (`tcgen05.cp/UTCCP`)
 - Issued by one thread in the warpgroup(s), equivalent to `mma`

TensorCore 5th Generation Family Instructions

Profiling Results: Matrix Layout for 128x256x16 fp16 with fp32 accumulate

SASS:

```
# A/B loaded, D allocated
UTCHMMA gdesc[UR4], gdesc[UR6], tmem[UR10], ...
```

Matrix A:

- Size: 128x16 x 2 bytes = 4096 bytes
- Stored in shared memory of the SM executing the warpgroup(s)
- Descriptor specifies location, format, swizzle, ...

Matrix B:

- Size: 16x256 x 2 bytes = 8192 bytes
- Stored in shared memory of the SM executing the warpgroup(s)
- Descriptor specifies location, format, swizzle, ...

Matrix D:

- Size: 128x256 x 4 bytes = 131,072 bytes, 256 columns in TMEM
- Input matrix for accumulation
- Output matrix

Summary for 1 warpgroup (4 warps):

- 1 warp instruction executed, 1 predicated-on thread executed instruction
- 1,048,576 ops = 128 x 256 x (16 multiplications + 15 additions) in fp16 + 128 x 256 additions in fp32

Source Page:

#	Label	Address	Source	Instructions Executed	Avg. Predicated-On Threads Executed
58		00007125 73721090	R2UR UR10, R2	1	1
59	.L_x_6	00007125 737210a0	@P0 ELECT P1, URZ, PT	1	1
60		00007125 737210b0	UTCHMMA gdesc[UR4], gdesc[UR6], tmem[UR10], tmem[UR12]	1	1
61		00007125 737210c0	@P1 PLOP3.LUT P0, PT, P1, PT, PT, 0x8, 0x80	1	1
62		00007125 737210d0	PLOP3.LUT P1, PT, PT, PT, PT, 0x8, 0x80	1	1

smsp__inst_executed_pipe_tensor_subpipe_hmma_op_hmma.sum

Tensor Core Roofline Table:

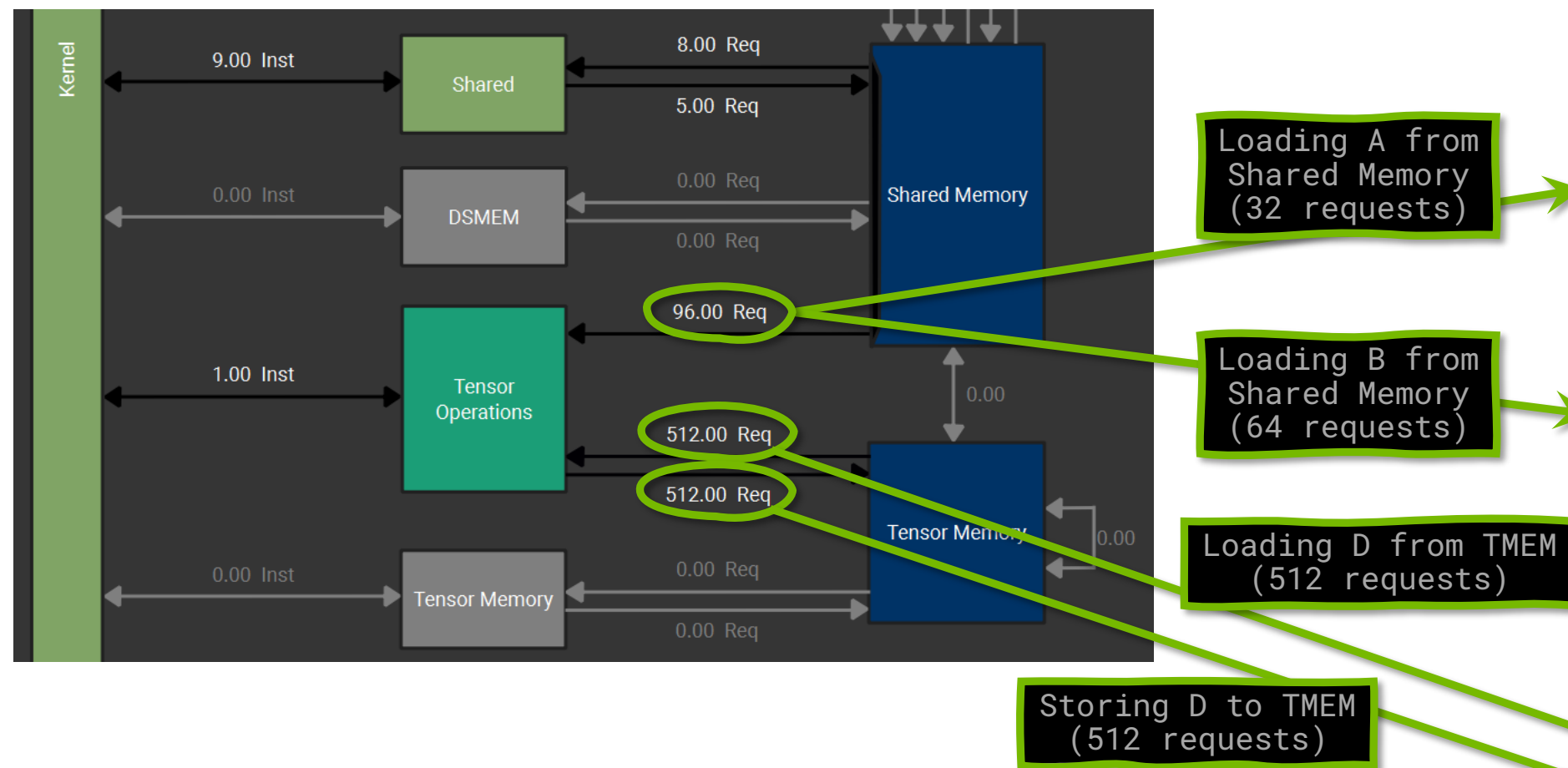
	# Operations	# Operations / Cycle	# Operations / s	Peak %	Peak Operations / Cycle	Peak Operations / s
Op:utchmma Src:bf16 Dst:fp32 Sparsity:off	0	0	0	0	1,212,416	2,049,004,859,813,084.25
Op:utchmma Src:bf16 Dst:fp32 Sparsity:on	0	0	0	0	2,424,832	4,098,009,719,626,168.50
Op:utchmma Src:fp16 Dst:fp16 Sparsity:off	0	0	0	0	1,212,416	2,049,004,859,813,084.25
Op:utchmma Src:fp16 Dst:fp16 Sparsity:on	0	0	0	0	2,424,832	4,098,009,719,626,168.50
Op:utchmma Src:fp16 Dst:fp32 Sparsity:off	1,048,576	181.21	306,242,990,654.21	0.01	1,212,416	2,049,004,859,813,084.25
Op:utchmma Src:fp16 Dst:fp32 Sparsity:on	0	0	0	0	2,424,832	4,098,009,719,626,168.50

sm__ops_path_tensor_op_utchmma_src_fp16_dst_fp32_sparsity_off.sum

TensorCore 5th Generation Family Instructions

Profiling Results: Matrix Layout for 128x256x16 fp16 with fp32 accumulate

Memory Chart:



SASS:

```
# A/B loaded, D allocated
UTCHMMA gdesc[UR4], gdesc[UR6], tmem[UR10], ...
```

Matrix A:

- Size: $128 \times 16 \times 2$ bytes = 4096 bytes
- Stored in shared memory of the SM executing the warpgroup(s)
- Descriptor specifies location, format, swizzle, ...

► **Matrix B:**

- Size: $16 \times 256 \times 2$ bytes = 8192 bytes
- Stored in shared memory of the SM executing the warpgroup(s)
- Descriptor specifies location, format, swizzle, ...

Matrix D:

- Size: $128 \times 256 \times 4$ bytes = 131,072 bytes, 256 columns in TMEM
- Input matrix for accumulation
- Output matrix

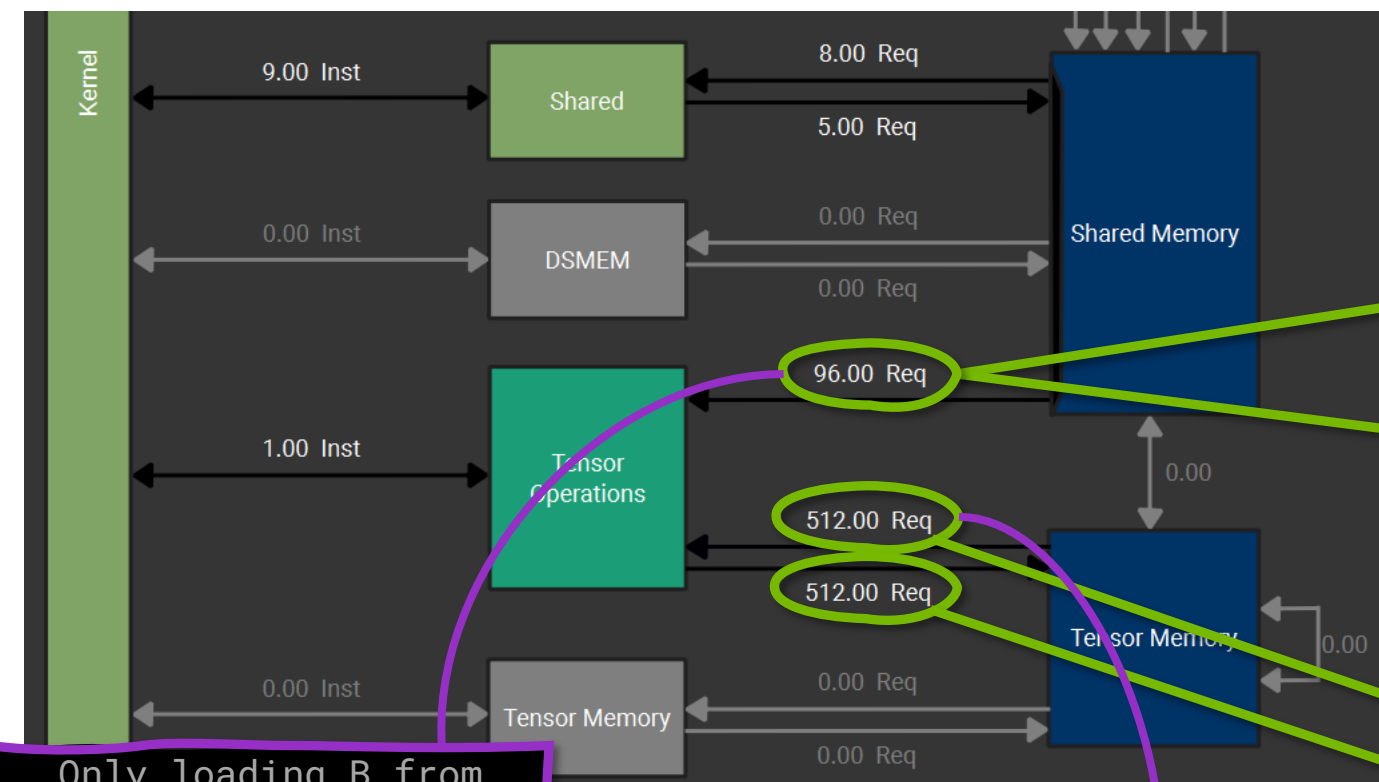
Summary for 1 warpgroup (4 warps):

- 1 warp instruction executed, 1 predicated-on thread executed instruction
- 1,048,576 ops = 128 x 256 x (16 multiplications + 15 additions) in fp16 + 128 x 256 additions in fp32

TensorCore 5th Generation Family Instructions

Profiling Results: Matrix Layout for 128x256x16 fp16 with fp32 accumulate

Memory Chart:



Only loading B from
Shared Memory
(-32 Requests, -33.33%)

Loading A from
Shared Memory
(32 requests)

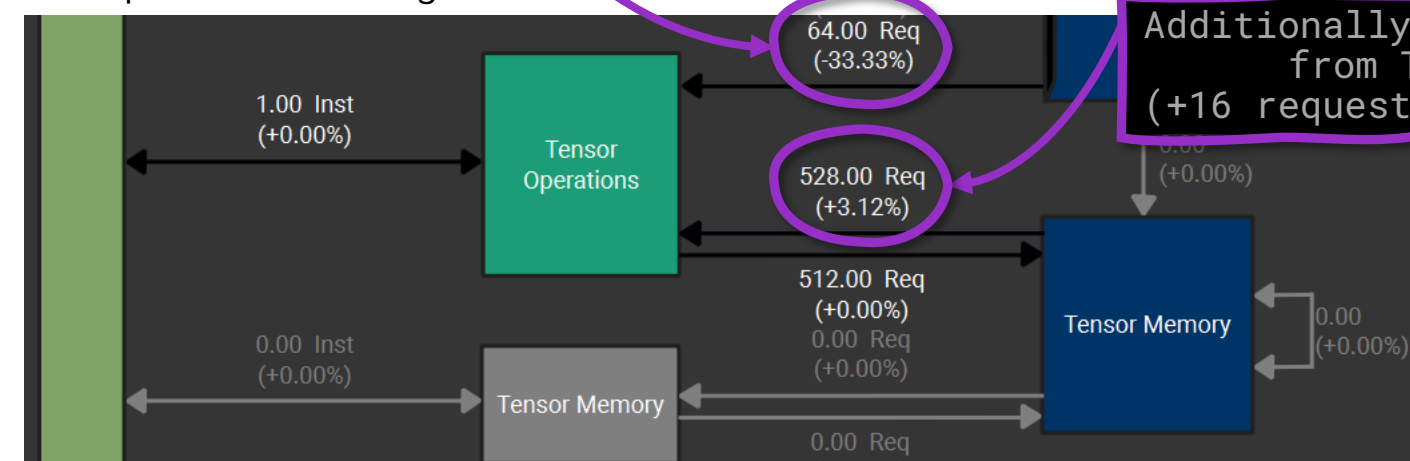
Loading B from
Shared Memory
(64 requests)

Loading D from TMEM
(512 requests)

Storing D to TMEM (512 requests)

Additionally Loading A
from TMEM
(+16 requests, +3.12%)

Comparison to loading A from TMEM:



SASS:

```
# A/B loaded, D allocated
UTCHMMA gdesc[UR4], gdesc[UR6], tmem[UR10], ...
```

Matrix A:

- Size: $128 \times 16 \times 2$ bytes = 4096 bytes
- Stored in shared memory of the SM executing the warpgroup(s)
- Descriptor specifies location, format, swizzle, ...

► **Matrix B:**

- Size: $16 \times 256 \times 2$ bytes = 8192 bytes
- Stored in shared memory of the SM executing the warpgroup(s)
- Descriptor specifies location, format, swizzle, ...

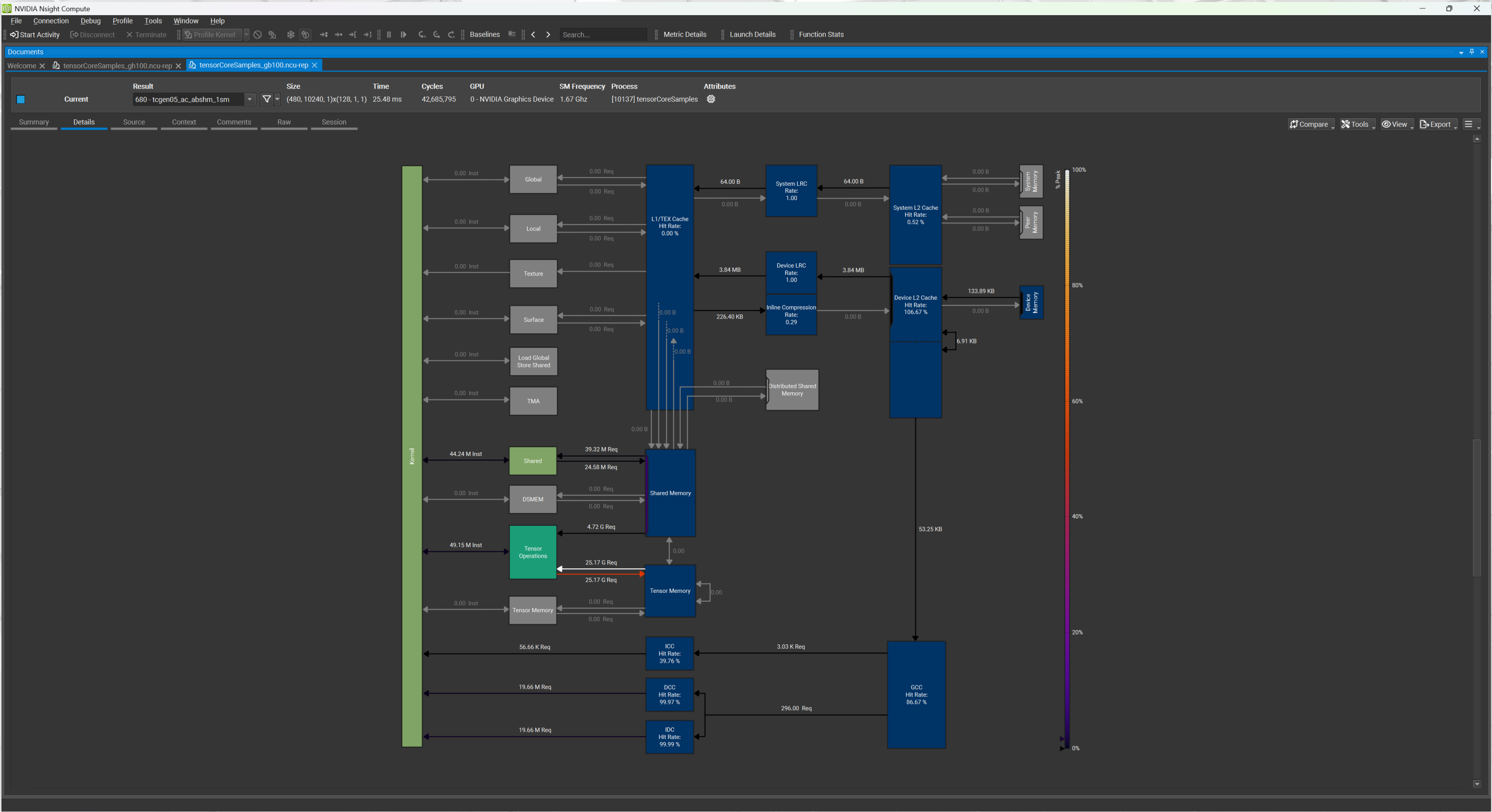
Matrix D:

- Size: $128 \times 256 \times 4$ bytes = 131,072 bytes, 256 columns in TMEM
- Input matrix for accumulation
- Output matrix

Summary for 1 warpgroup (4 warps):

- 1 warp instruction executed, 1 predicated-on thread executed instruction
- 1,048,576 ops = 128 x 256 x (16 multiplications + 15 additions) in fp16 + 128 x 256 additions in fp32

TensorCore 5th Generation Family Instructions



CuTe / CUTLASS

Overview

CUTLASS: CUDA Templates for Linear Algebra Subroutines

- Open-source C++ template library for high-performance GEMM and related computations on NVIDIA GPUs
- Header-only; ships as reusable, modular components that can be specialized via custom tiling sizes, data types, and algorithmic policies

CuTe: CUDA C++ template library for tensor abstractions (since CUTLASS 3.0)

- Provides hierarchically multidimensional **Layouts** and **Tensors** that handle complex indexing automatically
- Layout algebra lets you compose, tile, and partition layouts
- The programmer focuses on algorithm logic, not index math
- **CuTe DSL** as efficient Python abstraction (since CUTLASS 4.0)

GitHub (download)	https://github.com/NVIDIA/cutlass
Documentation	https://docs.nvidia.com/cutlass/latest/
CuTe Quickstart	https://docs.nvidia.com/cutlass/latest/media/docs/cpp/cute/00_quickstart.html
Examples / Tutorials	https://github.com/NVIDIA/cutlass/tree/main/examples

CuTe / CUTLASS

Example

```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

CuTe / CUTLASS

Example

Allocate/Release/Free TMEM

```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

CuTe / CUTLASS

Example

Partition global tensors
into per-CTA tiles {

Main Loop over 4 Tiles

Stream 4 Tiles through Shared
Each Tile processes subset of A * B

Allocate/Release/Free TMEM

```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

CuTe / CUTLASS

Example

Partition global tensors
into per-CTA tiles {

Setup TMA Descriptors {

TMA Copy of Tile Partition
of matrix A and B
Global → Shared {

Main Loop over 4 Tiles
Stream 4-Tiles through Shared
Each Tile processes subset of A * B {

Allocate/Release/Free TMEM

```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

CuTe / CUTLASS

Example

Partition global tensors
into per-CTA tiles {

Setup TMA Descriptors {

Main Loop over 4 Tiles
Stream 4 Tiles through Shared
Each Tile processes subset of A * B

**TMA Copy of Tile Partition
of matrix A and B**
Global → Shared {

MMAs in 4 Blocks
A, B from Shared
Output C → TMEM {
Blocks enable Shared reuse

Allocate/Release/Free TMEM

```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

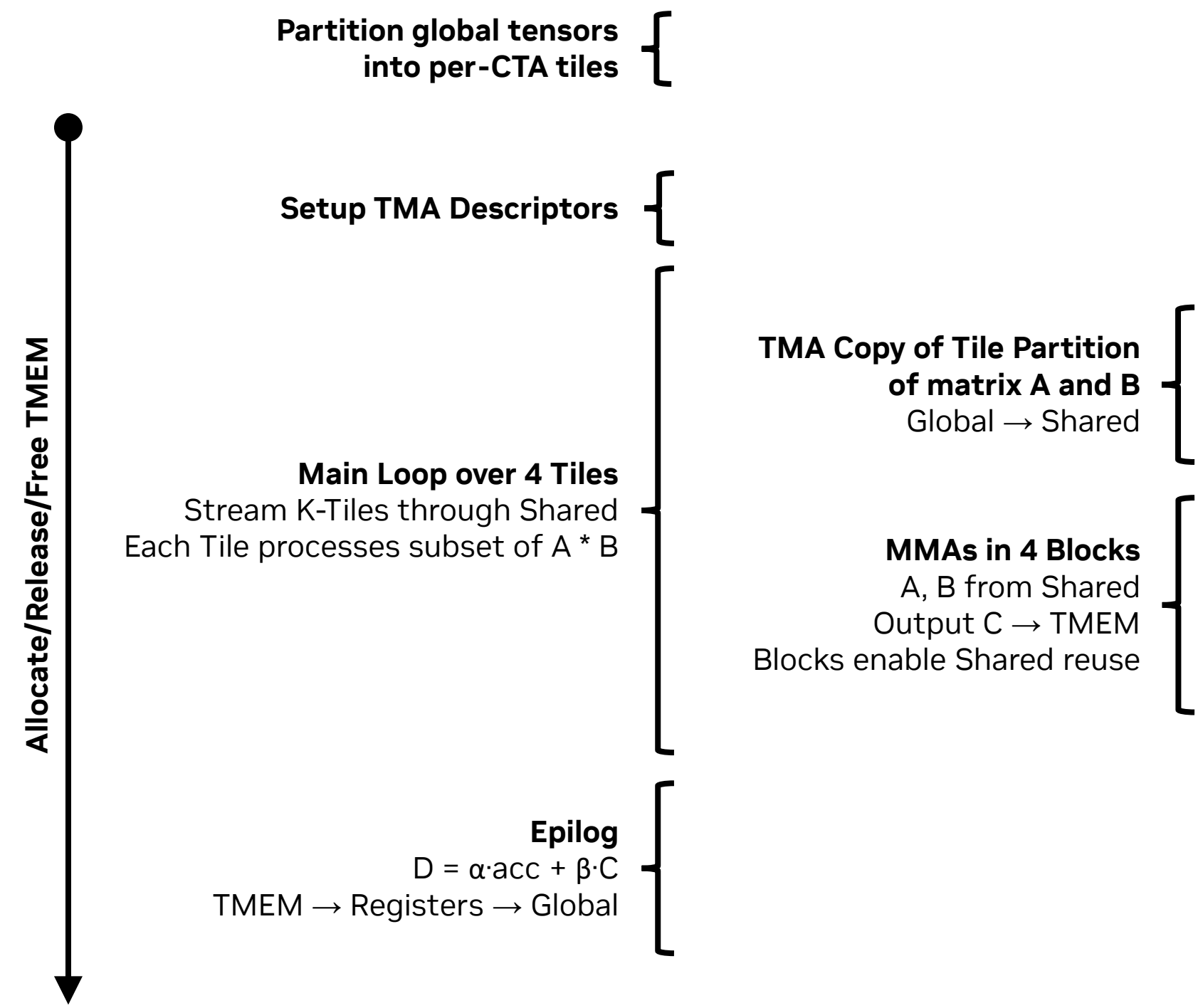
        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

CuTe / CUTLASS

Example



```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

CuTe / CUTLASS

Profile Results

Calculate:

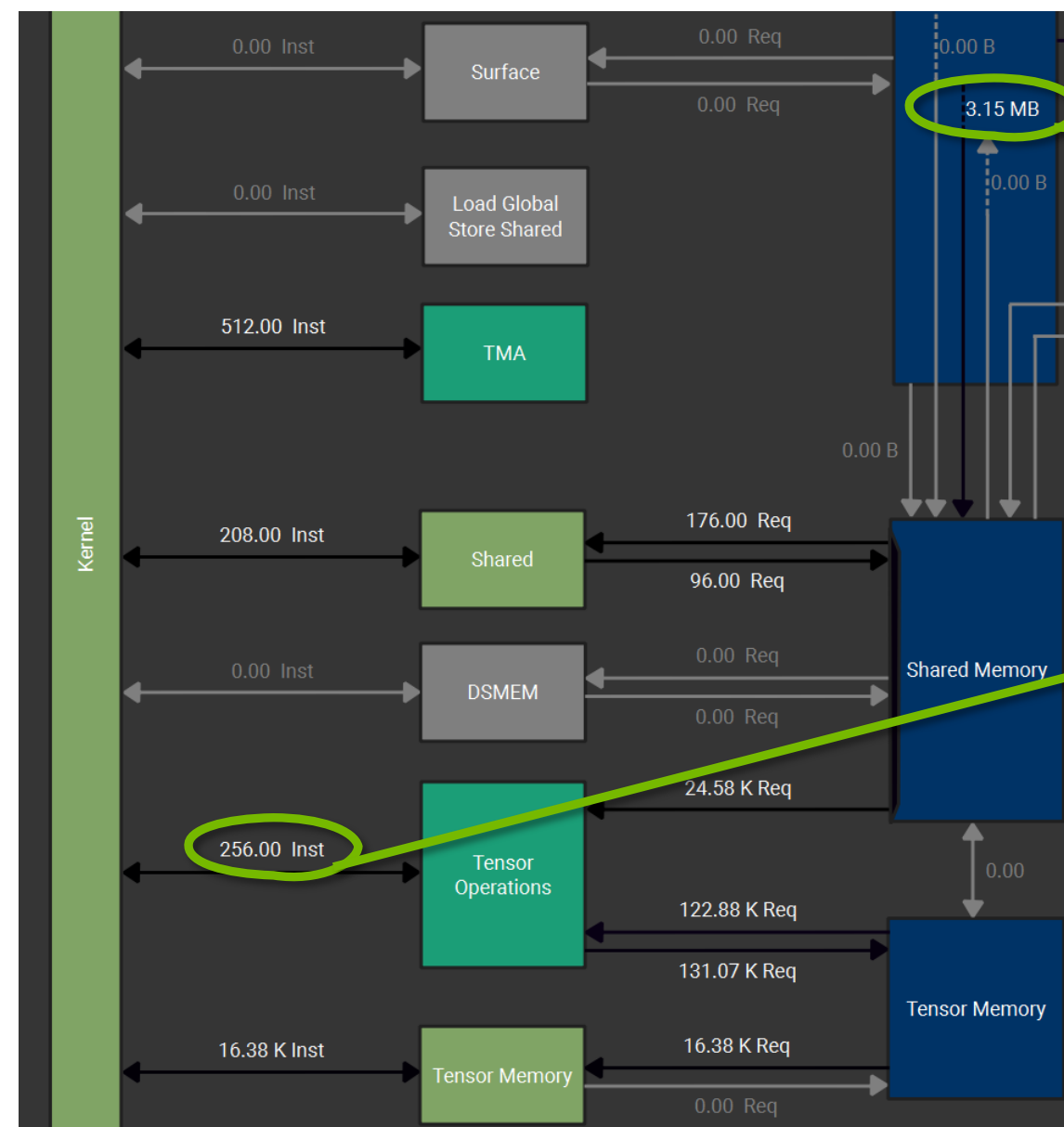
Problem Size: 512x1024x256, A,B: f16 → C,D: f32

Tile: Size: 128x256 (→ launching 16 CTAs)

TMA: A: 128 x 64 x 2 → 16KB, B: 64 x 256 x 2 → 32KB

MMA: 16 MMAs of size 128x256x16

```
cute_gemm_device<<<dim3(4, 4, 1), dim3(128, 1, 1), ... >>>
```



TMA: Global → Shared
16CTAs x 4Tiles * (16KB+32KB)
= 3,072KB

MMA:
16CTAs x 4Tiles x 4Blocks
= 256 Inst

256Inst x 128x256x16x2
= 268,435,456 ops

```
__global__ void cute_gemm_device(/* tensors mA, mB, mC, mD, tiled_mma, tma_atoms, ... */)
{
    Tensor gA = local_tile(mA, mma_tiler, mma_coord, Step<_1, X,_1>{});
    Tensor gB = local_tile(mB, mma_tiler, mma_coord, Step< X,_1,_1>{});

    if (elect_one_warp) tmem_allocator.allocate(..., &shared.tmem_base_ptr);

    auto [tAgA, tAsA] = tma_partition(tma_atom_A, ..., sA, gA);
    auto [tBgB, tBsB] = tma_partition(tma_atom_B, ..., sB, gB);

    for (int k = 0; k < 4; ++k) {
        if (elect_one_warp) {
            copy(tma_atom_A.with(tma_barrier), tAgA(_,k), tAsA);
            copy(tma_atom_B.with(tma_barrier), tBgB(_,k), tBsB);
        }
        wait_barrier(tma_barrier);

        if (elect_one_warp && elect_one_thread) {
            for (int kb = 0; kb < 4; ++kb) {
                gemm(tiled_mma, frag_A(_,_,kb), frag_B(_,_,kb), accum);
            }
            umma_arrive(&mma_barrier);
        }
        wait_barrier(mma_barrier);
    }

    copy(tmem_copy, accum, reg);
    axpby(alpha, reg, beta, C);
    copy(reg, gD);

    __syncthreads();
    if (elect_one_warp) {
        tmem_allocator.release_allocation_lock();
        tmem_allocator.free(shared.tmem_base_ptr, ...);
    }
}
```

Summary & Outlook



Nsight Compute

Tensor Cores

What have we covered?

- Mental model of all 3 types of Tensor Core operations with correlation to obtained profiling data
- Tensor Core Profiling in Nsight Compute:
 - **Tensor Core Roofline:** Architecture/Executed TOPs, utilized formats, percentage-of-peak performance
 - **PM Sampling:** Time series of Tensor pipe active (identify gaps/starvation), math pipe active, and data movements to identify pipelining/balancing issues
 - **Memory Chart:** Overview of data movement to and from Tensor Cores, identify bottlenecks in data movement
 - **Source Page:** Per-line correlation to your user-written kernel code, including PC sampling, stall reasons, ... (compile with `-lineinfo` to get correlation)

What have we not covered yet?

- Sparse MMA variants
 - Supported in Ampere and newer architectures
 - 2:4 structured sparsity effectively doubling compute throughput by encoding which 2 elements out of 4 consecutive elements are zero in a matrix
- Warp Specialization
 - Extending the idea of the CuTe example using `elect_one_warp`; assigning dedicated roles to each warp for further overlap between data management and compute

NVIDIA Nsight Compute

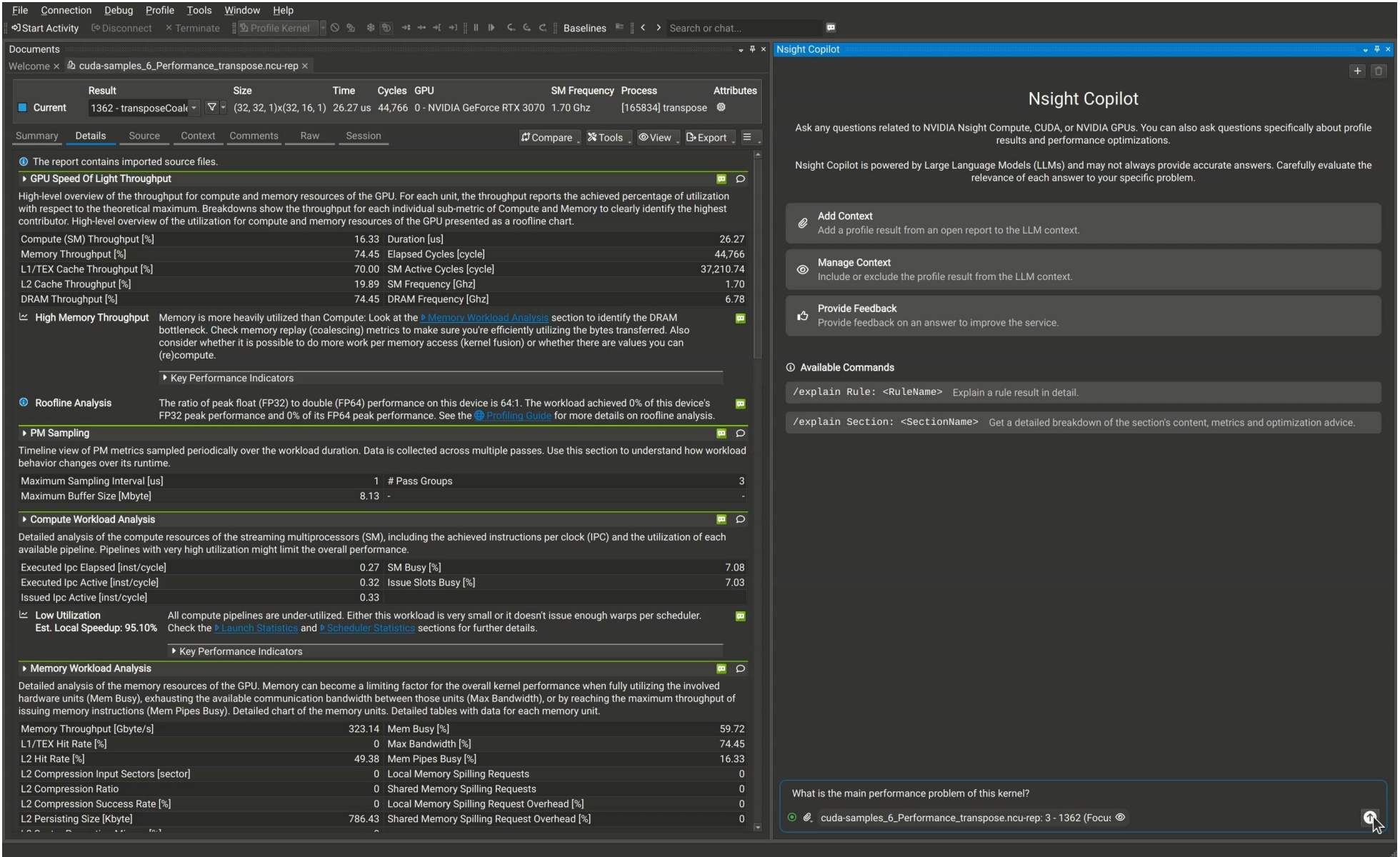
CoPilot Preview

AI-Powered Assistant within Nsight Compute:

- Trained to answer general profile information about:
 - Tools usage, metric descriptions, rule explanations, ...
 - Faster learning curve to become a GPU profiling expert
- Optional access to collected profile information
 - Find and understand bottlenecks of your own kernels

Give it a try starting with CUDA 13.2 (available now)

- Download from:
<https://developer.nvidia.com/tools-overview/nsight-compute/get-started>
(version installed with CUDA toolkit does not include the CoPilot Preview)
- Requirements:
 - Supported on Windows, Linux, Mac, and SBSA
 - Python 4.13 or newer
 - Login with NVIDIA Developer Account
- Future Work:
 - More agentic tools
 - More training material
 - Self-Hosting capability



Developer Tools Across GTC

Sessions

- [S81590](#): Lowering the Barriers: How New Tools Unlock GPU Acceleration for Everyone
- [S81831](#): AI Coding for the GPU: Building a Coding Agent to Help GPU developers write Speed of Light Code
- [S81653](#): LLM-Generated CUDA Kernels: Are We There Yet?
- [S81772](#): Don't leave Tensors on the Table: Programming and Optimizing Tensor Cores

Labs

- [DLIT81545](#): Profiling Python and AI workloads with Nsight Compute
- [DLIT81641](#): Find the Bottleneck – Optimize AI pipelines with Nsight Systems
- [DLIT81579](#): Optimizing PyTorch models for high-performance inference with Nsight Deep Learning Designer

Connect with the Experts

- [CWES81549](#): What's in Your Developer Toolbox? CUDA, AI, and Graphics Profiling, Optimization, and Debugging Tools
- [CWES81535](#): CUDA Developer Best Practices
- [CWES81771](#): How to Run and Optimize Your Workloads on the NVIDIA Grace and Vera CPUs
- [QA82372](#): What's in Your Developer Toolbox? CUDA, AI, and HPC, Optimization, and Debugging Tools

Live Demos

Come and visit the Developer Tools pod at the NVIDIA booth during exhibition floor hours!

Developer Tools are **free** and packaged in the latest version of the CUDA Toolkit

<https://developer.nvidia.com/cuda-downloads>

Support is available via:

<https://forums.developer.nvidia.com/c/developer-tools>

More information at:

<https://developer.nvidia.com/tools-overview>

